LangChain 마스터 — 강사용 상세 대본

- 슬라이드 수: 328

- 부제: LangChain v1 실전 에이전트 개발

- 기준: source-repo/02_langchain/*.ipynb

---

01. AI 에이전트 개발 기초

안녕하세요. AI 에이전트 개발 기초 과정을 시작하겠습니다. 이번 트랙의 중심은 create_agent를 중심으로 모델, 도구, 메시지, 메모리, 미들웨어, MCP, 가드레일을 하나의 운영 가능한 에이전트 인터페이스로 연결하는 일입니다. 슬라이드는 개념의 흐름을 잡기 위한 자료이고, 뒤쪽 코드 부록은 실제 노트북의 실행 셀을 순서대로 확인하기 위한 자료입니다. 강의 중에는 API 이름을 외우는 것보다, 그 API가 어떤 문제를 해결하려고 등장했는지에 집중해 주세요. 이 기준을 잡으면 이후 실습에서 코드가 길어져도 실행 흐름을 놓치지 않습니다.

---

02. 오리엔테이션

이제 PART 01 오리엔테이션으로 넘어갑니다. 입문에서 만든 첫 에이전트를 기준점으로 삼고, LangChain v1의 모델·도구·메모리·운영 기능을 한 단계씩 확장합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다.

---

03. 이번 트랙은 첫 에이전트를 프로덕션 에이전트로 키우는 흐름입니다

이번 트랙은 첫 에이전트를 프로덕션 에이전트로 키우는 흐름입니다. 각 노트북은 새 프레임워크를 도입하기보다, 같은 LangChain 에이전트에 필요한 능력을 하나씩 붙입니다. 흐름을 왼쪽에서 오른쪽으로 읽으면 MODEL 단계에서는 messages를 다룹니다. TOOLS 단계에서는 schemas를 다룹니다. STATE 단계에서는 memory를 다룹니다. CONTROL 단계에서는 middleware를 다룹니다. RUN 단계에서는 production을 다룹니다. LangChain 트랙의 핵심 질문은 에이전트가 무엇을 맡길 일인가가 아니라 운영 가능한 에이전트를 어떻게 설계하나입니다. 에이전트 수업에서 중요한 것은 각 단계를 별개의 기능으로 보는 것이 아니라, 입력과 출력이 다음 단계로 어떻게 넘어가는지 보는 것입니다.

---

04. 02_langchain은 13개 노트북으로 에이전트 생명주기를 전부 훑습니다

02_langchain은 13개 노트북으로 에이전트 생명주기를 전부 훑습니다. 노트북 순서는 구현 편의가 아니라 운영 성숙도 순서입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 기초 조립은 01~04 소개·Quickstart·모델·도구입니다. 판단 기준은 모델, 메시지, 도구, 구조화 출력을 갖춘 기본 에이전트입니다. 상태와 제어는 05~07 메모리·미들웨어·HITL입니다. 판단 기준은 대화 상태, 런타임 컨텍스트, 사람 승인, 안전한 중단/재개입니다. 복합 패턴은 08~09 멀티에이전트·워크플로·RAG입니다. 판단 기준은 서브에이전트, 핸드오프, 라우터, StateGraph, 검색 보강입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

05. LangChain v1 기준점

이제 PART 02 LangChain v1 기준점으로 넘어갑니다. LangChain v1은 create_agent()를 중심으로 모델·도구·상태·미들웨어를 한 실행 그래프에 묶습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/01_introduction.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

06. LangChain은 단일 API가 아니라 세 레이어가 맞물린 실행 계층입니다

LangChain은 단일 API가 아니라 세 레이어가 맞물린 실행 계층입니다. 노트북의 3계층 그림은 LangChain·LangGraph·Deep Agents가 왜 함께 언급되는지 설명합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 LangChain은 에이전트 생성의 기준 API입니다. create_agent, @tool, ChatOpenAI가 여기에 속합니다. LangGraph는 LangChain 에이전트의 내부 실행 엔진입니다. 상태 그래프·체크포인터·스트리밍이 이 계층에서 제공됩니다. Deep Agents는 사전 구축된 장기 작업 하네스입니다. 빠른 프로토타이핑에 적합하지만, 기초 인터페이스는 LangChain을 공유합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

07. create_agent()의 기본 동작은 ReAct 루프입니다

create_agent()의 기본 동작은 ReAct 루프입니다. 모델은 바로 답하지 않고, 필요하면 도구를 호출하고 결과를 관찰한 뒤 다시 판단합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Reasoning은 사용자 메시지를 보고 지금 답할지, 도구를 쓸지 모델이 판단합니다. Acting은 도구 호출이 필요하면 스키마에 맞춰 인자를 만들고 도구를 실행합니다. Observation은 도구 결과가 ToolMessage로 돌아오면 모델이 다음 행동이나 최종 답변을 결정합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

08. v1의 기준 API는 create_agent, InMemorySaver, middleware입니다

v1의 기준 API는 create_agent, InMemorySaver, middleware입니다. 예전 v0.x의 여러 체인·메모리 객체보다, v1은 에이전트 중심 인터페이스로 수렴합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 에이전트 생성은 create_react_agent() 등 여러 경로입니다. 판단 기준은 from langchain.agents import create_agent입니다. 실행 엔진은 AgentExecutor 중심입니다. 판단 기준은 내부적으로 LangGraph 그래프로 실행입니다. 메모리는 ConversationBufferMemory 계열입니다. 판단 기준은 InMemorySaver 같은 체크포인터입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

09. 설치 확인은 네 가지 import 통과 여부로 시작합니다

설치 확인은 네 가지 import 통과 여부로 시작합니다. 노트북은 실습 시작 전에 create_agent, tool, ChatOpenAI, InMemorySaver를 기준 모듈로 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 에이전트는 create_agent가 모든 실습의 기본 실행 단위입니다. 도구는 @tool은 함수명을 도구 이름으로, docstring을 도구 설명으로 씁니다. 상태는 InMemorySaver는 멀티턴 대화를 저장하는 첫 체크포인터입니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 agents에서 create_agent; tools에서 tool를 준비합니다. 핵심 API는 create_agent, ChatOpenAI, InMemorySaver입니다. 노트북 기준 위치는 01_introduction.ipynb 핵심 import 확인입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

10. Quickstart — 첫 에이전트

이제 PART 03 Quickstart — 첫 에이전트로 넘어갑니다. @tool 함수와 create_agent()를 연결하고, invoke·stream·thread_id·context를 한 번에 체험합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/02_quickstart.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

11. 함수에 @tool을 붙이면 모델이 호출할 수 있는 도구가 됩니다

함수에 @tool을 붙이면 모델이 호출 가능한 도구가 됩니다. docstring과 타입 힌트는 사람이 읽는 주석이 아니라 모델이 인자를 고르는 스키마입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 함수명은 기본 도구 이름이 됩니다. 짧고 동사형으로 둡니다. docstring는 모델이 도구의 용도를 이해하는 핵심 설명입니다. 타입 힌트는 도구 호출 인자를 정수·문자열 같은 명확한 타입으로 제한합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 add, multiply는 실제 작업 단위입니다. 노트북 기준 위치는 02_quickstart.ipynb @tool 기본입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

12. create_agent()는 모델·도구·시스템 프롬프트를 하나의 그래프로 묶습니다

create_agent()는 모델·도구·시스템 프롬프트를 하나의 그래프로 묶습니다. 생성된 객체는 내부적으로 LangGraph 그래프이며 invoke와 stream을 바로 제공합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 model은 노트북 기준 기본 모델은 ChatOpenAI(model='gpt-5.4')입니다. tools는 도구 리스트는 에이전트가 선택 가능한 행동 공간입니다. system_prompt는 도구를 언제 어떻게 쓸지에 대한 운영 규칙입니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent, invoke입니다. 노트북 기준 위치는 02_quickstart.ipynb create_agent + invoke입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

13. stream_mode='updates'는 에이전트의 단계별 움직임을 보여줍니다

stream_mode='updates'는 에이전트의 단계별 움직임을 다룹니다. 최종 답만 보는 invoke와 달리, stream은 모델 노드와 도구 노드의 업데이트를 순서대로 확인하게 해줍니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 화면에서 진행 표시는 도구 호출과 최종 응답을 실시간 로그처럼 보여줄 수 있습니다. 디버깅은 어느 노드에서 잘못된 인자가 만들어졌는지 확인하기 쉽습니다. UI 연결은 이후 프론트엔드 스트리밍의 기본 감각이 됩니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 stream입니다. 노트북 기준 위치는 02_quickstart.ipynb stream updates입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

14. 멀티턴 대화는 checkpointer와 thread_id가 함께 있어야 유지됩니다

멀티턴 대화는 checkpointer와 thread_id가 함께 있어야 유지됩니다. 체크포인터는 상태를 저장하고, thread_id는 어떤 대화 세션을 이어갈지 지정합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 checkpointer은 InMemorySaver()가 에이전트 상태를 메모리에 보관합니다. thread_id는 동일한 thread_id를 쓰면 이전 메시지를 이어받습니다. 격리는 다른 thread_id는 완전히 별도 대화로 취급됩니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 memory에서 InMemorySaver를 준비합니다. 핵심 API는 create_agent, InMemorySaver, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 02_quickstart.ipynb InMemorySaver + thread_id입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

15. context_schema는 호출별 메타데이터를 메시지 이력과 분리합니다

context_schema는 호출별 메타데이터를 메시지 이력과 분리합니다. 사용자 ID·locale·권한 같은 값은 messages에 섞지 말고 context로 넣는 편이 안전합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 상태와 분리는 사용자 메타데이터가 대화 히스토리를 오염시키지 않습니다. 타입 안정성은

dataclass로 런타임 컨텍스트 모양을 고정합니다. 이름 규칙은 에이전트 name은 노트북처럼 snake_case가 트레이싱과 라우팅에 깔끔합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 dataclasses에서 dataclass를 준비합니다. 클래스 Context는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다. 노트북 기준 위치는 02_quickstart.ipynb context_schema입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

16. Tavily 검색 도구를 붙이면 에이전트가 최신 정보에 직접 접근합니다

Tavily 검색 도구를 붙이면 에이전트가 최신 정보에 직접 접근합니다. TAVILY_API_KEY가 있을 때만 실행되는 선택 실습으로, 웹 검색 결과를 도구 호출 흐름에 그대로 연결합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 검색 도구는 TavilySearch(max_results=3)는 에이전트가 그대로 호출 가능한 검색 도구입니다. 선택 실행은 TAVILY_API_KEY 유무로 셀 실행을 분기해 키가 없어도 노트북이 끊기지 않습니다. 조합은 검색 도구도 다른 도구와 동일하게 tools 리스트에 넣어 create_agent에 전달합니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 create_agent, invoke입니다. 노트북 기준 위치는 02_quickstart.ipynb TavilySearch 연동 (선택)입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

17. 모델과 메시지

이제 PART 04 모델과 메시지로 넘어갑니다. 프로바이더가 달라도 invoke·stream·batch와 메시지 타입을 일관되게 다루는 법을 익힙니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/03_models_and_messages.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

18. init_chat_model()은 프로바이더 문자열 하나로 모델 초기화를 통일합니다

init_chat_model()은 프로바이더 문자열 하나로 모델 초기화를 통일합니다. OpenAI는 ChatOpenAI를 직접 써도 되고, 여러 프로바이더를 바꾸려면 init_chat_model이 편합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 OpenAI는 openai:gpt-5.4입니다. 판단 기준은 langchain-openai + OPENAI_API_KEY입니다. Anthropic는 anthropic:claude-sonnet-4-6입니다. 판단 기준은 langchain-anthropic + ANTHROPIC_API_KEY입니다. Google는 google:gemini-2.5-flash-lite입니다. 판단 기준은 langchain-google-genai + GOOGLE_API_KEY입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

---

19. 모델 호출은 invoke·stream·batch 세 패턴으로 나뉩니다

모델 호출은 invoke·stream·batch 세 패턴으로 나뉩니다. 같은 모델 객체라도 사용자가 기다리는 방식과 처리량 요구에 따라 호출법이 달라집니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 invoke()는 AIMessage입니다. 판단 기준은 단일 입력에 대한 완성 응답이 필요할 때입니다. stream()는 Iterator[AIMessageChunk]입니다. 판단 기준은 채팅 UI처럼 토큰 단위 응답을 보여줄 때입니다. batch()는 List[AIMessage]입니다. 판단 기준은 여러 입력을 병렬로 처리하고 max_concurrency를 걸 때입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

20. 메시지 타입은 대화의 역할과 도구 결과를 명확히 분리합니다

메시지 타입은 대화의 역할과 도구 결과를 명확히 분리합니다. 문자열 하나만 넘기기보다 메시지 리스트를 쓰면 시스템 지시·사용자 입력·도구 결과를 안정적으로 재현합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 SystemMessage는 시스템 지시입니다. 판단 기준은 모델의 톤·정책·도구 사용 기준을 정합니다. HumanMessage는 사용자 입력입니다. 판단 기준은 사용자가 방금 보낸 질문입니다. AIMessage는 모델 응답입니다. 판단 기준은 텍스트뿐 아니라 tool_calls와 usage_metadata를 가질 수 있습니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

21. 운영 모델은 출력 포맷·재시도·타임아웃을 초기화 시점에 고정합니다

운영 모델은 출력 포맷·재시도·타임아웃을 초기화 시점에 고정합니다. content_blocks와 usage metadata를 표준화하면 멀티모달·툴콜·비용 추적 코드가 단순해집니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 output_version은 output_version='v1' 또는 LC_OUTPUT_VERSION=v1로 표준 직렬화를 켭니다. Usage callback는 UsageMetadataCallbackHandler로 호출 단위 토큰·캐시 통계를 모읍니다. 회복력은 max_retries와 timeout은 프로덕션 기본값입니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 langchain_openai에서 ChatOpenAI; callbacks에서 UsageMetadataCallbackHandler를 준비합니다. 핵심 API는 ChatOpenAI, invoke입니다. 노트북 기준 위치는 03_models_and_messages.ipynb 운영 모델 설정입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

22. 텍스트와 이미지를 한 메시지에 함께 담을 수 있습니다

텍스트와 이미지를 한 메시지에 함께 담을 수 있습니다. HumanMessage의 content에 텍스트·이미지 블록을 리스트로 나열하면 비전 모델이 두 입력을 동시에 해석합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 콘텐츠 블록은 content를 리스트로 두고 type: text, type: image_url 블록을 나열합니다. 이미지 전달은 image_url은 URL 또는 base64로 전달하며, 비전을 지원하는 모델에서만 동작합니다. 표준 출력은 output_version='v1'을 지정하면 응답이 content_blocks로 표준화되어 텍스트·이미지·툴콜이 분리 노출됩니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 messages에서 HumanMessage; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI, invoke입니다. 노트북 기준 위치는 03_models_and_messages.ipynb 멀티모달 메시지 (이미지 입력)입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

23. config 하나로 트레이스 식별과 동시 실행 상한을 함께 겁니다

config 하나로 트레이스 식별과 동시 실행 상한을 함께 겁니다. config 파라미터는 콜백뿐 아니라 트레이싱 메타데이터, batch 동시성 제한까지 함께 전달합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 화면에서 run_name / tags는 run_name은 트레이스에서 이 호출을 식별하고, tags는 필터링 가능한 라벨을 붙입니다. metadata는 metadata에 임의의 키-값 컨텍스트(실험명, 사용자 ID 등)를 실어 트레이스에 남깁니다. max_concurrency는 batch() 호출에서 max_concurrency로 동시 실행 상한을 정해 토큰 폭주를 막습니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 invoke입니다. 노트북 기준 위치는 03_models_and_messages.ipynb Runtime Config입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

24. 프롬프트 캐싱은 긴 prefix를 반복 호출할 때 비용과 지연을 줄입니다

프롬프트 캐싱은 긴 prefix를 반복 호출할 때 비용과 지연을 줄입니다. 노트북은 implicit caching과 explicit caching을 함께 설명합니다. 묶음별로 읽으면 Implicit caching. 프로바이더가 동일 prefix를 자동 감지합니다. 여기서는 OpenAI gpt-5.4 계열처럼 호출자가 별도 cache block을 작성하지 않아도 되는 흐름입니다; 코드 변경이 적어 빠르게 켤 수 있지만, 캐시 단위 제어는 프로바이더에 맡깁니다를 봅니다. Explicit caching. 호출자가 캐시 단위를 명시합니다. 여기서는 Anthropic 계열은 prompt_cache_key나 cache_control 블록으로 캐시 경계를 지정합니다; 캐시 hit 여부는 usage_metadata.input_token_details.cache_read 같은 필드로 추적합니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

25. 도구와 구조화 출력

이제 PART 05 도구와 구조화 출력으로 넘어갑니다. 함수에 @tool을 붙이고 Pydantic으로 입력 스키마를 정의한 뒤, 도구 실행 결과를 구조화된 출력으로 연결하는 흐름을 봅니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/04_tools_and_structured_output.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

26. Pydantic args_schema는 복잡한 도구 입력을 모델에게 명확히 설명합니다

Pydantic args_schema는 복잡한 도구 입력을 모델에게 명확히 설명합니다. 단순 타입 힌트로 부족하면 Field 설명과 기본값을 도구 스키마에 포함합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 Field 설명은 각 파라미터가 무엇인지 모델에게 전달합니다. 기본값은 모델이 매번 모든 인자를 채우지 않아도 안전한 값을 씁니다. 스키마 확인은 args_schema.model_json_schema()로 모델에게 보이는 형태를 점검합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 pydantic에서 BaseModel, Field; tools에서 tool를 준비합니다. 클래스 SearchQuery는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다. 노트북 기준 위치는 04_tools_and_structured_output.ipynb Pydantic 도구 스키마입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

27. ToolRuntime은 도구 안에서 현재 state와 실행 메타데이터를 꺼내는 통로입니다

ToolRuntime은 도구 안에서 현재 state와 실행 메타데이터를 꺼내는 통로입니다. 도구 인자로 받으면 안 되는 내부 정보는 ToolRuntime으로 주입받습니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 runtime.state는 현재 메시지 이력과 에이전트 상태를 읽습니다. execution_info는 thread_id, run_id, attempt 같은 디버깅 키를 얻습니다. server_info는 LangGraph Platform 같은 호스팅 환경에서는 assistant_id, user 정보가 들어올 수 있습니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 tools에서 tool, ToolRuntime를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_user_info는 실제 작업 단위입니다. 노트북 기준 위치는 04_tools_and_structured_output.ipynb ToolRuntime입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

28. Command를 반환하는 도구는 ToolMessage를 함께 넣어야 합니다

Command를 반환하는 도구는 ToolMessage를 함께 넣어야 합니다. 상태 업데이트만 하고 ToolMessage를 빠뜨리면 모델은 도구 응답을 못 받은 것으로 보고 같은 호출을 반복할 가능성이 있습니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 Command.update는 메시지 밖의 state 필드를 갱신합니다. ToolMessage는 도구 호출 ID와 짝을 맞춰 모델에게 실행 결과를 돌려줍니다. runtime.tool_call_id는 현재 tool_call과 ToolMessage를 연결하는 키입니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 types에서

Command; messages에서 ToolMessage를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 remember_user_pref는 실제 작업 단위입니다. 노트북 기준 위치는 04_tools_and_structured_output.ipynb Command + ToolMessage입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

29. 구조화 출력은 모델 직접 방식과 에이전트 response_format 방식이 있습니다

구조화 출력은 모델 직접 방식과 에이전트 response_format 방식이 있습니다. 모델만 쓸 때는 with_structured_output, 에이전트 최종 답을 강제할 때는 ToolStrategy나 ProviderStrategy를 씁니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 직접 모델은 model.with_structured_output(MySchema)는 에이전트 없이 바로 스키마 객체를 돌려줍니다. ToolStrategy는 도구 호출 메커니즘으로 구조화하므로 모든 모델에서 동작하고 안정적입니다. ProviderStrategy는 지원 모델(OpenAI·Claude·Grok)의 네이티브 구조화 출력이라 더 빠르고 정확합니다. strict 옵션은 langchain>=1.2부터 지원됩니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 structured_output에서 ToolStrategy; dataclasses에서 dataclass를 준비합니다. 클래스 CalculationResult는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다. 두 코드 블록을 비교하면 ToolStrategy. 모든 모델 블록은 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. ProviderStrategy. 네이티브 지원 모델 블록은 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

30. 메모리와 스트리밍

이제 PART 06 메모리와 스트리밍으로 넘어갑니다. 대화를 저장하는 방법과 실행 상태를 실시간으로 보여주는 방법을 실제 운영 관점에서 구분합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/05_memory_and_streaming.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

31. 단기 메모리는 checkpointer가 저장하고 thread_id가 대화를 고릅니다

단기 메모리는 checkpointer가 저장하고 thread_id가 대화를 고릅니다. 같은 에이전트라도 thread_id가 다르면 서로 다른 대화로 격리됩니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 대화 상태는 체크포인터는 메시지 히스토리와 실행 상태를 저장합니다. 세션 키는 configurable.thread_id가 복원할 대화를 지정합니다. 개발용은 InMemorySaver는 프로세스가 꺼지면 사라지므로 실서비스는 영속 저장소가 필요합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 memory에서 InMemorySaver를 준비합니다. 핵심 API는 create_agent, InMemorySaver, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 05_memory_and_streaming.ipynb 단기 메모리입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

32. 단기 메모리와 장기 메모리는 저장 대상과 접근 방식이 다릅니다

단기 메모리와 장기 메모리는 저장 대상과 접근 방식이 다릅니다. 대화 이력은 checkpointer가 자동으로, 사용자 선호도는 store를 도구에서 명시적으로 다룹니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 범위는 하나의 thread_id 안입니다. 판단 기준은 여러 세션·여러 thread를 가로지름입니다. 저장 대상은 메시지 히스토리와 그래프 상태입니다. 판단 기준은 사용자 선호도, 프로필, 학습 데이터입니다. 접근 방식은 에이전트가 자동 복원입니다. 판단 기준은 도구가 runtime.store로 명시 접근입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

33. 장기 메모리는 namespace에 user_id를 넣어 사용자별로 격리합니다

장기 메모리는 namespace에 user_id를 넣어 사용자별로 격리합니다. store를 create_agent에 넘기지 않으면 도구 내부 runtime.store는 None이므로 도구가 실패합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 namespace는 ("preferences", user_id)처럼 튜플로 다층 격리를 만듭니다. context는 사용자 ID는 메시지에서 파싱하지 말고 runtime.context에서 받습니다. tool 인자로 넘기지 않는 이유는 LLM에 노출되는 도구 시그니처를 깨끗하게 유지하고, 멀티테넌트 격리를 에이전트 작성자가 중앙에서 통제하기 위해서입니다. 영속화는 운영 환경에서는 langgraph-checkpoint-postgres 계열 저장소를 검토합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 dataclasses에서 dataclass; memory에서 InMemoryStore를 준비합니다. 클래스 Context는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다. 노트북 기준 위치는 05_memory_and_streaming.ipynb 장기 메모리입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

34. 긴 대화는 trim_messages를 before_model 미들웨어로 연결해 줄입니다

긴 대화는 trim_messages를 before_model 미들웨어로 연결해 줄입니다. 대화를 무한히 누적하면 비용과 지연이 늘어나므로, 모델 호출 직전에 토큰 예산에 맞게 잘라냅니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 화면에서 strategy='last'는 최근 메시지를 우선 유지합니다. include_system은 시스템 메시지는 항상 보존합니다. request.override는 모델에 들어가는 메시지만 교체하고 원래 state 구조는 유지합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 messages에서 trim_messages; middleware에서 before_model를 준비합니다. 데코레이터 before_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 trim_context는 실제 작업 단위입니다. 노트북 기준 위치는 05_memory_and_streaming.ipynb 메시지 트리밍입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

35. stream_mode는 무엇을 실시간으로 보여줄지 결정합니다

stream_mode는 무엇을 실시간으로 보여줄지 결정합니다. 채팅 UI, 디버그 패널, 진행률 이벤트는 같은 스트리밍이라도 필요한 데이터가 다릅니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 values는 각 단계의 전체 상태입니다. 판단 기준은 디버깅과 상태 추적입니다. updates는 각 노드의 변경분입니다. 판단 기준은 에이전트 진행 상황 표시입니다. messages는 토큰 단위 메시지입니다. 판단 기준은 채팅 버블 렌더링입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

36. v2 스트리밍은 GraphOutput과 특수 청크 규칙을 함께 이해해야 합니다

v2 스트리밍은 GraphOutput과 특수 청크 규칙을 함께 이해해야 합니다. 노트북은 백엔드 표준 패턴으로 version='v2'와 여러 모드 동시 수신을 다룹니다. 묶음별로 읽으면 GraphOutput. dict처럼 보지 말고 속성으로 접근합니다. 여기서는 result.value에 최종 상태가 들어갑니다; result.interrupts에 HITL 중단 요청이 들어갑니다를 봅니다 tool_calls 누적. 마지막 청크까지 기다립니다 여기서는 토큰 스트리밍 중 tool call 인자는 부분 JSON으로 나뉘어 옵니다; chunk_position == 'last'가 보일 때 완전한 인자 형태라고 판단합니다를 봅니다 custom mode. 도구가 진행률을 직접 흘립니다 여기서는 도구 내부에서 get_stream_writer()를 호출해 검색 시작·완료 같은 이벤트를 보냅니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

37. 미들웨어 파이프라인

이제 PART 07 미들웨어 파이프라인으로 넘어갑니다. 모델 호출 전후, 모델 호출 래핑, 도구 호출 래핑, 동적 프롬프트를 한 구조로 이해합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/06_middleware.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

38. 미들웨어는 모델과 도구 사이에 운영 로직을 주입합니다

미들웨어는 모델과 도구 사이에 운영 로직을 주입합니다. 입력 검증, 요약, 폴백, 캐싱, 로깅, 가드레일은 에이전트 본문보다 미들웨어로 분리하는 편이 좋습니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 before_model은 모델 호출 전에 메시지를 검증하거나 수정합니다. wrap_model_call은 모델 호출 자체를 감싸 재시도·폴백·캐싱을 구현합니다. wrap_tool_call은 도구 실행 전후에 권한·로깅·에러 처리를 넣습니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

39. 빌트인 미들웨어는 운영에서 반복되는 패턴을 바로 제공합니다

빌트인 미들웨어는 운영에서 반복되는 패턴을 바로 제공합니다. 직접 훅을 쓰기 전에 노트북의 기본 카탈로그부터 확인합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 SummarizationMiddleware는 긴 대화 요약입니다. 판단 기준은 trigger=('messages', 10)처럼 발동 조건 지정입니다. ModelFallbackMiddleware는 모델 장애 시 폴백입니다. 판단 기준은 primary 실패 후 backup 모델 순차 호출입니다. ContextEditingMiddleware는 오래된 tool_use 제거입니다. 판단 기준은 trigger=('tokens', 100_000), keep 규칙입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

40. before_model과 after_model은 모델 호출 주변의 관측 지점입니다

before_model과 after_model은 모델 호출 주변의 관측 지점입니다. 입력 로깅과 출력 로깅은 에이전트 도구 로직을 건드리지 않고도 붙일 수 있습니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 화면에서 before_model은 모델에 전달될 메시지 개수, 입력 정책 위반을 확인합니다. after_model은 모델 응답 내용과 tool_calls를 기록하거나 후처리합니다. 조합은 여러 미들웨어를 리스트 순서대로 넣어 파이프라인을 만듭니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 before_model; middleware에서 after_model를 준비합니다. 데코레이터 before_model, after_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 log_model_input, log_model_output는 실제 작업 단위입니다. 두 코드 블록을 비교하면 입력 로깅 블록은 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 출력 로깅 블록은 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

41. wrap_model_call은 request.override로 모델 요청을 안전하게 바꿉니다

wrap_model_call은 request.override로 모델 요청을 안전하게 바꿉니다. ModelRequest를 직접 mutate하지 않고 새 요청을 만들어 handler에 넘기는 것이 노트북의 권장 패턴입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 변경 가능 필드는 messages, tools, system_message, response_format을 override합니다. 폴백과 제한은 비로그인 사용자의 도구를 제한하거나 모델 호출 재시도를 구현합니다. 불변성은 원본 요청을 건드리지 않아 미들웨어 조합이 안전해집니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 wrap_model_call, ModelRequest, ModelResponse를 준비합니다. 데코레이터 wrap_model_call가

일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 restrict_tools_for_anonymous는 실제 작업 단위입니다. 노트북 기준 위치는 06_middleware.ipynb request.override입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

42. wrap_tool_call은 handler 앞뒤로 도구 실행을 감쌉니다

wrap_tool_call은 handler 앞뒤로 도구 실행을 감쌉니다. wrap_model_call이 모델 호출을 감싸는 것과 동일한 패턴을 도구 실행에 적용해 타이밍·로깅·에러 처리를 끼워 넣습니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 ToolCallRequest는 request.tool_call['name'], request.tool_call['args']로 실행 대상 도구와 인자를 읽습니다. handler 호출은 handler(request)를 호출해야 실제 도구가 실행되며, 호출 전후로 시간을 재면 도구별 성능을 측정합니다. 예외 처리는 도구 실행이 실패하면 예외를 잡아 로깅한 뒤 다시 던지거나, ToolMessage로 변환해 에이전트에 안전하게 알릴 수 있습니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 wrap_tool_call; time를 준비합니다. 데코레이터 wrap_tool_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 tool_timing_logger는 실제 작업 단위입니다. 노트북 기준 위치는 06_middleware.ipynb handler(request) 전후 훅입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

43. dynamic_prompt는 요청마다 시스템 프롬프트를 새로 만듭니다

dynamic_prompt는 요청마다 시스템 프롬프트를 새로 만듭니다. 현재 시각, 사용자 역할, 테넌트 정책처럼 매 호출 달라지는 지침을 프롬프트에 반영합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 런타임 지침은 동일 에이전트라도 요청 컨텍스트에 따라 다른 시스템 메시지를 씁니다. A/B 테스트는 프롬프트 변형을 미들웨어 레벨에서 통제합니다. 역할 전환은 후반의 핸드오프 패턴에서도 역할별 프롬프트 교체에 활용됩니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 dynamic_prompt; datetime에서 datetime를 준비합니다. 데코레이터 dynamic_prompt가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 add_datetime_context는 실제 작업 단위입니다. 노트북 기준 위치는 06_middleware.ipynb dynamic_prompt입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

44. wrap_tool_call은 상태 스키마가 필요한 정책과 분리해서 설계합니다

wrap_tool_call은 모든 경우에 상태 스키마가 필요한 것은 아닙니다. 단순히 도구 실행을 감싸서 로깅하거나 예외를 ToolMessage로 바꿀 때는 기존 wrapper 패턴이면 충분합니다. 다만 도구 호출 이력을 별도 상태 필드에 남기려면 state_schema를 선언하고, 반환값도 messages와 커스텀 상태 업데이트를 함께 맞춰야 합니다.

---

45. 프로바이더 설정은 공통 모델 옵션과 벤더 전용 옵션을 분리합니다

프로바이더 설정은 공통 모델 옵션과 벤더 전용 옵션을 분리해서 설명합니다. OpenAI의 prompt_cache_options나 Responses 계열 옵션, Google의 reasoning_effort와 thinking_config는 create_agent 자체보다 모델 초기화와 호출 설정의 회귀 검증 대상입니다. 수업에서는 옵션 이름보다 어디서 검증해야 하는지를 남깁니다.

---

46. Generative UI와 AG-UI는 코어 에이전트와 별도 통합 표면입니다

Generative UI와 AG-UI는 프론트엔드 통합 표면입니다. 지금 트랙의 스트리밍 수업은 messages, tool calls, interrupt를 UI로 투영하는 기본 계약을 다룹니다. React 컴포넌트 생성이나 AG-UI 이벤트 프로토콜은 별도 실행환경이 필요하므로 코어 에이전트 설명에 섞지 않고 통합 트랙으로 연결합니다.

47. 가드레일 훅은 예외보다 jump_to='end'로 안전하게 끝내는 편이 좋습니다

가드레일 훅은 예외보다 jump_to='end'로 안전하게 끝내는 편이 좋습니다. 위험 입력을 감지했을 때 사용자에게 안내 메시지를 남기고 그래프를 종료합니다. 묶음별로 읽으면 @hook_config(can_jump_to=[...])로 허용 점프 대상을 선언합니다. 여기서는 end. 즉시 종료하고 안내 메시지를 남깁니다; tools. 도구 노드로 점프합니다; model. 모델 노드로 재진입합니다를 봅니다. 비동기 환경에서는 a- 프리픽스 훅을 씁니다. 여기서는 abefore_model, aafter_model, awrap_model_call, adynamic_prompt를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

48. HITL과 런타임 컨텍스트

이제 PART 08 HITL과 런타임 컨텍스트로 넘어갑니다. 위험한 도구 호출 앞에서 사람이 승인·거절하도록 하는 HITL(Human-in-the-Loop) 패턴과, 도구에 실행 정보를 전달하는 ToolRuntime을 살펴봅니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/07_hitl_and_runtime.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

49. HITL은 위험한 도구 호출 직전에 그래프를 멈춥니다

HITL은 위험한 도구 호출 직전에 그래프를 멈춥니다. 이메일 전송·파일 삭제·결제처럼 되돌리기 어려운 행동은 모델 판단만으로 실행하지 않습니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 제안은 모델이 도구 호출과 인자를 만듭니다. 중단은 HumanInTheLoopMiddleware가 interrupt를 발생시킵니다. 재개는 사람의 decision을 Command(resume=...)로 넣어 실행을 이어갑니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

50. HumanInTheLoopMiddleware는 checkpointer와 함께 써야 중단 후 재개됩니다

HumanInTheLoopMiddleware는 checkpointer와 함께 써야 중단 후 재개됩니다. 중단 상태를 저장할 체크포인터가 없으면 승인 뒤 같은 지점에서 이어갈 수 없습니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 interrupt_on은 도구별 승인 필요 여부와 허용 decision을 지정합니다. checkpointer은 InMemorySaver가 중단된 그래프 상태를 보관합니다. thread_id는 재개할 때도 같은 thread_id를 사용해야 합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 HumanInTheLoopMiddleware; memory에서 InMemorySaver를 준비합니다. 핵심 API는 create_agent, InMemorySaver, HumanInTheLoopMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 07_hitl_and_runtime.ipynb HITL setup입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

51. Command(resume)의 decision은 approve·edit·reject·respond 네 가지입니다

Command(resume)의 decision은 approve·edit·reject·respond 네 가지입니다. decision 리스트 순서는 interrupt의 action_requests 순서와 맞아야 하고, edit로 인자를 바꿀 때는 보수적으로 수정해야 합니다. 원래 인자를 크게 바꾸면 모델이 접근 방식을 재평가해 도구를 여러 번 실행합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 approve는 도구를 그대로 실행입니다. 판단 기준은 모델이 만든 인자를 승인합니다. edit는 도구 인자를 수정 후 실행입니다. 판단 기준은 수신자·금액·파일명 같은 위험 인자를 고칩니다. reject는 도구 실행을 거부입니다. 판단 기준은 거부 사유가 모델에게 전달됩니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

52. resume은 같은 config로 Command를 다시 invoke하는 두 단계 패턴입니다

resume은 같은 config로 Command를 다시 invoke하는 두 단계 패턴입니다. 첫 invoke에서 interrupts를 확인하고, 두 번째 invoke에서 사람이 내린 결정을 넣습니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 화면에서 1단계는 도구 호출 제안에서 그래프가 멈추고 result.interrupts가 생깁니다. 2단계는 Command(resume={...})를 같은 config로 넣어 재개합니다. GraphOutput는 v2 결과는 result.value와 result.interrupts로 봅니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 invoke, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 07_hitl_and_runtime.ipynb Command resume입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

53. ToolRuntime[T]는 사용자 컨텍스트와 실행 메타데이터를 도구에 주입합니다

ToolRuntime[T]는 사용자 컨텍스트와 실행 메타데이터를 도구에 주입합니다. 권한 확인, 감사 로그, 멀티테넌트 분기는 도구 인자 대신 런타임 정보로 처리합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 화면에서 context_schema는 호출자가 넣는 사용자 컨텍스트 타입을 고정합니다. execution_info는 thread_id, run_id, attempt를 로깅에 씁니다. server_info는 배포 환경에서는 assistant_id, graph_id, user를 권한 분리에 활용합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 dataclasses에서 dataclass; tools에서 tool, ToolRuntime를 준비합니다. 클래스 UserContext는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다. 노트북 기준 위치는 07_hitl_and_runtime.ipynb ToolRuntime context입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

54. 멀티에이전트 패턴

이제 PART 09 멀티에이전트 패턴으로 넘어갑니다. 여러 에이전트를 두는 다섯 가지 방식 중 요구사항에 맞는 패턴을 고르고, 불필요한 오버헤드를 피하는 법을 봅니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/08_multi_agent.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

55. 멀티에이전트 패턴은 라우팅 주체와 상태 공유 방식이 다릅니다

멀티에이전트 패턴은 라우팅 주체와 상태 공유 방식이 다릅니다. 에이전트를 여러 개 만든다고 항상 좋아지는 것은 아닙니다. 단순 작업은 single agent가 기준입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Subagents는 메인 에이전트입니다. 판단 기준은 독립 작업을 반복·병렬로 위임하고 결과만 취합입니다. Handoffs는 도구 호출·상태 전환입니다. 판단 기준은 대화 상태를 다음 역할로 넘기는 순차 멀티홉입니다. Skills는 단일 에이전트입니다. 판단 기준은 프롬프트를 바꿔 하나의 에이전트가 여러 역할 수행입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

56. Subagents는 감독자가 전문 에이전트를 도구처럼 호출하는 패턴입니다

Subagents는 감독자가 전문 에이전트를 도구처럼 호출하는 패턴입니다. 서브에이전트의 내부 추론은 격리되고, 메인 에이전트는 최종 결과만 받아 컨텍스트를 아낍니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 격리는 각 subagent 호출은 독립 컨텍스트에서 실행됩니다. 반복 작업은 검색·분석을 여러 번 호출해 결과를 모으는 작업에 적합합니다. local history는 checkpointer=True로 서브에이전트만의 히스토리를 유지합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

57. 서브에이전트가 많아지면 노출 방식도 설계해야 합니다

서브에이전트가 많아지면 노출 방식도 설계해야 합니다. 도구 슬롯이 많아질수록 모델이 무엇을 호출해야 하는지 헷갈립니다. 묶음별로 읽으면 단일 dispatch 도구로 묶는 세 가지 방식에서는 System prompt 열거. 10개 미만일 때 단순합니다; Literal 인자 제약. 10~30개 규모에서 tool schema가 enum을 강제합니다; Tool-based discovery. 30개 이상이면 list_subagents와 describe_subagent로 동적 탐색합니다를 봅니다. 장시간 서브 작업은 3-tool 패턴으로 비동기화합니다. 여기서는 start_job. 작업 제출 후 job_id 반환; check_status. pending/running/done/failed 확인; get_result. 완료 결과 회수를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

58. Handoff는 결과 위임이 아니라 대화 상태 전환입니다

Handoff는 결과 위임이 아니라 대화 상태 전환입니다. 고객지원처럼 사용자가 같은 대화에서 부서가 넘어간 느낌을 받아야 할 때 handoff를 씁니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 상태 전달은 메시지 히스토리가 다음 역할로 전달됩니다. Command(goto)는 그래프 노드 간 전환을 명시합니다. 단일 에이전트 대안은 노트북은 wrap_model_call로 role에 따라 프롬프트·도구를 교체하는 방식을 권장합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

59. Router는 분류 후 Send로 여러 도메인에 fan-out(병렬 분산)할 때 강해집니다

Router는 분류 후 Send로 여러 도메인에 fan-out(병렬 분산)할 때 강해집니다. 단일 분기만 한다면 일반 조건부 엣지와 큰 차이가 없습니다. 진짜 장점은 병렬 분배입니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 분류는 입력을 math, code, general 같은 도메인으로 나눕니다. Send는 여러 노드에 동시에 payload를 보내 병렬 처리합니다. 취합은 도메인별 결과를 fan-in하여 최종 답변을 만듭니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

60. Skills는 에이전트를 그대로 두고 프롬프트만 갈아 끼웁니다

Skills는 에이전트를 그대로 두고 프롬프트만 갈아 끼웁니다. 별도 에이전트나 그래프 전환 없이, 도구 호출로 전문 지침을 그때그때 불러오는 점진적 공개 패턴입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 단일 에이전트는 skill_agent 하나가 번역·요약·코딩 역할을 모두 처리합니다. 점진적 공개는 스킬을 한꺼번에 시스템 프롬프트에 넣지 않고, load_skill 호출 시점에만 지침을 노출합니다. 프롬프트 교체는 역할 전환이 상태 전달(Handoffs)이나 분류(Router) 없이 도구 반환값 교체로 일어납니다. 코드는 다음 지점을 중심으로 읽습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 load_skill는 실제 작업 단위입니다. 핵심 API는 create_agent 입니다. 노트북 기준 위치는 08_multi_agent.ipynb load_skill로 프롬프트만 교체입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

61. 멀티에이전트 선택 기준은 독립성·상태 전달·역할 전환입니다

멀티에이전트 선택 기준은 독립성·상태 전달·역할 전환입니다. 처음부터 복잡한 팀 구조로 가지 말고, 질문 세 개로 패턴을 좁힙니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 에이전트가 독립적으로 작업 가능한가는 Subagents입니다. 판단 기준은 상태 전달 필요성 확인입니다. 대화 상태가 다음 역할로 넘어가야 하는가는 Handoffs입니다. 판단 기준은 Skills 또는 Router 검토입니다. 하나의 에이전트가 프롬프트만 바꾸면 되는가는 Skills입니다. 판단 기준은 Custom StateGraph입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

62. 커스텀 워크플로와 RAG

이제 PART 10 커스텀 워크플로와 RAG로 넘어갑니다. StateGraph로 흐름을 직접 설계하는 법과, RAG(Retrieval-Augmented Generation, 검색 증강 생성)로 검색 결과를 답변에 반영하는 법을 연결합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/09_custom_workflow_and_rag.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

63. StateGraph는 노드·엣지·상태로 실행 흐름을 명시합니다

StateGraph는 노드·엣지·상태로 실행 흐름을 명시합니다. create_agent가 감춰 주던 흐름을 직접 그려야 할 때 LangGraph의 StateGraph를 사용합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 State는 노드들이 공유하는 데이터 구조입니다. 노트북은 TypedDict로 정의합니다. Node는 상태를 받아 일부 필드를 업데이트하는 함수입니다. Edge는 다음에 실행할 노드를 정하는 연결입니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

64. 기본 StateGraph는 상태 타입을 정하고 노드를 연결한 뒤 compile합니다

기본 StateGraph는 상태 타입을 정하고 노드를 연결한 뒤 compile합니다. 노드는 전체 상태를 다시 만들지 않고 바뀐 필드만 dict로 반환합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 화면에서 TypedDict는 상태 키와 타입을 명시합니다. add_node는 처리 함수를 그래프 노드로 등록합니다. compile는 빠진 연결·도달 불가 노드 같은 구조 오류를 잡습니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 typing에서 TypedDict; graph에서 StateGraph, START, END를 준비합니다. 클래스 WorkflowState는 데이터 구조나 정책 단위를 정의합니다. 함수 preprocess, analyze는 실제 작업 단위입니다. 노트북 기준 위치는 09_custom_workflow_and_rag.ipynb StateGraph 기본입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

65. 조건부 엣지는 런타임 상태에 따라 다음 노드를 고릅니다

조건부 엣지는 런타임 상태에 따라 다음 노드를 고릅니다. 입력 분류 후 math·language·general 핸들러로 나누는 패턴은 라우터의 기초입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 화면에서 classify는 상태에 category를 기록합니다. route는 상태를 보고 다음 노드 이름을 반환합니다. mapping는 반환값을 실제 노드에 매핑합니다. 코드는 다음 지점을 중심으로 읽습니다. 함수 route는 실제 작업 단위입니다. 핵심 API는 add_conditional_edges입니다. 노트북 기준 위치는 09_custom_workflow_and_rag.ipynb conditional edges입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

66. 에이전트를 노드로 넣으면 리서치와 작성 파이프라인을 분리할 수 있습니다

에이전트를 노드로 넣으면 리서치와 작성 파이프라인을 분리합니다. 노트북의 research → writer 예제는 에이전트를 함수 노드처럼 연결하는 방법을 다룹니다. 흐름을 왼쪽에서 오른쪽으로 읽으면 USER 단계에서는 question을 다룹니다. RESEARCH 단계에서는 create_agent를 다룹니다. WRITER 단계에서는 create_agent를 다룹니다. END 단계에서는 report를 다룹니다. RAG의 Rewrite → Retrieve → Agent 3노드 패턴도 같은 생각입니다. 검색 친화 질의로 바꾸고, 문서를 가져오고, 에이전트가 답합니다. 에이전트 수업에서 중요한 것은 각 단계를 별개의 기능으로 보는 것이 아니라, 입력과 출력이 다음 단계로 어떻게 넘어가는지 보는 것입니다.

67. RAG는 검색과 생성을 결합하지만 구현 방식은 세 가지입니다

RAG는 검색과 생성을 결합하지만 구현 방식은 세 가지입니다. 처음에는 2단계로 시작하고, 검색 판단이 필요해지면 에이전틱 RAG로 확장합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 기본 2단계는 검색 → 생성입니다. 판단 기준은 단순하고 빠르며 디버깅이 쉽습니다. 에이전틱 RAG은 에이전트가 검색 도구 반복 호출입니다. 판단 기준은 질문이 모호하거나 검색 전략이 달라질 때 유연합니다. 하이브리드는 키워드 + 시맨틱 검색입니다. 판단 기준은 정확도와 재현율을 함께 챙깁니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

68. 긴 문서는 chunk_size와 overlap으로 잘라 임베딩에 맞춥니다

긴 문서는 chunk_size와 overlap으로 잘라 임베딩에 맞춥니다. RecursiveCharacterTextSplitter가 RAG 파이프라인의 첫 번째 실전 단계입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 화면에서 chunk_size는 chunk_size는 한 청크의 최대 문자 수로, 임베딩 모델의 컨텍스트 한도에 맞춰 조절합니다. chunk_overlap는 chunk_overlap은 인접 청크가 겹치는 구간으로, 문장이 경계에서 끊겨도 맥락 손실을 줄입니다. split_text는 문서별로 호출해 chunks 리스트에 누적하며, 이후 임베딩·검색의 입력 단위가 됩니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 langchain_text_splitters에서 RecursiveCharacterTextSplitter를 준비합니다. 노트북 기준 위치는 09_custom_workflow_and_rag.ipynb chunk_size=200, chunk_overlap=50입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

69. 벡터 스토어 없이도 @tool 하나로 에이전트가 검색을 수행합니다

벡터 스토어 없이도 @tool 하나로 에이전트가 검색을 수행합니다. 단어 집합 교집합만으로 만든 simple_search를 retrieve_info 도구로 감싸 에이전트에 연결합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 simple_search는 쿼리·문서를 소문자 단어 집합으로 바꿔 교집합 크기를 점수로 사용합니다. 임베딩이 없어도 동작합니다. @tool retrieve_info는 검색 로직을 도구로 노출해 에이전트가 필요할 때 호출합니다. create_agent는 system_prompt로 항상 retrieve_info 호출을 강제해 근거 없는 답변을 방지합니다. 코드는 다음 지점을 중심으로 읽습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 simple_search, retrieve_info는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 노트북 기준 위치는 09_custom_workflow_and_rag.ipynb 단어 교집합 점수로 top_k=3 문서 반환입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

70. 임베딩과 FAISS를 연결하면 의미 기반 유사도 검색이 가능합니다

임베딩과 FAISS를 연결하면 의미 기반 유사도 검색이 가능합니다. 키워드가 겹치지 않아도 의미가 비슷한 청크를 찾아내는 임베딩 모델 + 벡터 스토어 조합입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 화면에서 OpenAIEmbeddings는 text-embedding-3-small 모델로 청크를 의미 벡터로 변환합니다. FAISS.from_texts는 청크와 임베딩 함수를 받아 벡터 스토어를 즉시 생성합니다. similarity_search는 쿼리를 벡터로 바꿔 k=3 최근접 청크를 반환합니다. 단어 매칭이 아닌 의미 매칭입니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 langchain_openai에서 OpenAIEmbeddings; vectorstores에서 FAISS를 준비합니다. 노트북 기준 위치는 09_custom_workflow_and_rag.ipynb text-embedding-3-small + FAISS, k=3 유사도 검색입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

71. LangChain RAG는 다섯 빌딩 블록으로 나누어 설계합니다

LangChain RAG는 다섯 빌딩 블록으로 나누어 설계합니다. 노트북은 간단 검색과 FAISS 예제로 검색 보강의 구조를 먼저 익힙니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Document Loaders는 PyPDFLoader, WebBaseLoader입니다. 판단 기준은 파일·웹·DB를 Document로 적재입니다. Text Splitters는 RecursiveCharacterTextSplitter입니다. 판단 기준은 긴 문서를 청크로 분할입니다. Embedding Models는 OpenAIEmbeddings입니다. 판단 기준은 텍스트를 의미 벡터로 변환입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

72. 프로덕션 운영

이제 PART 11 프로덕션 운영으로 넘어갑니다. 로컬 디버깅 도구부터 배포·평가·관측성까지, 에이전트를 운영에 올리기 위한 체크리스트를 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/10_production.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

73. 프로덕션 준비는 로컬 디버그에서 배포 후 관측성까지 이어집니다

프로덕션 준비는 로컬 디버그에서 배포 후 관측성까지 이어집니다. 에이전트가 한 번 잘 답하는 것과 계속 운영되는 것은 다른 문제입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 로컬 개발은 LangSmith Studio + langgraph dev입니다. 판단 기준은 노드·도구 호출·상태를 시각적으로 디버깅입니다. 테스트는 GenericFakeChatModel, pytest입니다. 판단 기준은 API 비용 없이 결정론적 흐름 검증입니다. 평가는 agentevals trajectory match입니다. 판단 기준은 도구 호출 순서와 포함 관계 검증입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

74. langgraph.json은 로컬 Studio와 배포가 읽는 그래프 진입점입니다

langgraph.json은 로컬 Studio와 배포가 읽는 그래프 진입점입니다. 노트북은 langgraph dev 서버와 LangSmith Studio(로컬 디버깅용 시각화 UI)를 연결해 핫 리로드·스레드 재실행을 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 화면에서 그래프 경로는 어떤 파일의 어떤 객체가 실행 그래프인지 지정합니다. 로컬 서버는 langgraph dev가 127.0.0.1:2024에서 서버를 띄웁니다. Studio는 Hosted Studio가 local baseUrl로 접속해 trace와 상태를 다룹니다. 노트북 기준 위치는 langgraph.json Studio 진입점입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

75. REST로 직접 호출할 때는 assistant_id·input·stream_mode를 페이로드에 담습니다

REST로 직접 호출할 때는 assistant_id·input·stream_mode를 페이로드에 담습니다. SDK 없이 curl만으로도 배포된 그래프를 그대로 스트리밍 호출합니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 화면에서 엔드포인트는 POST /runs/stream이 실행과 스트리밍을 함께 처리합니다. 페이로드는 assistant_id, input, stream_mode 세 필드가 핵심입니다. 배포 인증은 Platform deployment에서는 LangSmith API key를 헤더로 함께 보냅니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 stream입니다. 노트북 기준 위치는 10_production.ipynb REST로 직접 호출입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

76. GenericFakeChatModel은 API 호출 없이 에이전트 로직을 고정 테스트합니다

GenericFakeChatModel은 API 호출 없이 에이전트 로직을 고정 테스트합니다. CI에서는 실제 모델의 변동성보다 도구 호출과 상태 전이가 의도대로 일어나는지가 먼저입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 결정론은 항상 같은 메시지를 내므로 테스트가 흔들리지 않습니다. 비용 0은 외부 API 호출 없이 빠르게 돌릴 수 있습니다. 로직 검증은 프롬프트 품질이 아니라 도구 호출 경로와 파싱을 봅니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 language_models에서 GenericFakeChatModel; messages에서 AIMessage를 준비합니다. 핵심 API는 create_agent입니다. 노트북 기준 위치는 10_production.ipynb fake model test입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

77. trajectory 평가는 에이전트가 어떤 도구를 어떤 순서로 썼는지 검증합니다

trajectory 평가는 에이전트가 어떤 도구를 어떤 순서로 썼는지 검증합니다. 최종 답만 맞아도 위험한 도구를 잘못 썼다면 프로덕션 테스트는 실패해야 합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 strict는 도구가 같은 순서로 호출되어야 함입니다. 판단 기준은 정확한 프로세스가 중요한 워크플로입니다. unordered는 도구 집합만 같으면 순서 무관입니다. 판단 기준은 병렬·순서 비민감 작업입니다. subset는 기대 경로가 실제 경로에 포함입니다. 판단 기준은 추가 탐색은 허용하지만 필수 행동은 검증입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

78. 프로덕션 체크리스트는 보안과 운영 항목을 코드와 같은 수준으로 봅니다

프로덕션 체크리스트는 보안과 운영 항목을 코드와 같은 수준으로 봅니다. 테스트가 통과해도 키 관리·관측성·문서화가 없으면 운영 준비가 끝난 것이 아닙니다. 묶음별로 읽으면 출시 전에서는 단위 테스트와 트라젝토리 평가를 자동화합니다; API 키와 env를 배포 환경에서 주입하고 코드에 남기지 않습니다; 입력 검증과 가드레일을 미들웨어로 분리합니다를 봅니다. 출시 후에서는 LangSmith로 전체 실행 흐름, 토큰, 지연, 에러를 추적합니다; Agent Chat UI로 스트리밍·도구 호출·HITL을 수동 점검합니다; 장애 시 재현 가능한 thread와 trace를 남깁니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

79. MCP 도구 연결

이제 PART 12 MCP 도구 연결으로 넘어갑니다. MCP가 서버·클라이언트로 어떻게 나뉘는지, 로컬(stdio)과 원격(streamable-http) 연결을 어떻게 고르는지 봅니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/11_mcp.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

80. MCP는 서버·클라이언트·호스트가 역할을 나누는 도구 연결 규격입니다

MCP는 서버, 클라이언트, 호스트가 역할을 나누는 도구 연결 규격입니다. 최신 LangChain에서는 MCPAdapter가 서버 연결을 열고 list_tools로 LangChain 도구 목록을 가져옵니다. 여기서는 서버와 호스트를 분리해 운영 경계를 명확히 보는 것이 핵심입니다.

81. stdio는 로컬 개발용, streamable-http는 원격 MCP 서버용입니다

stdio는 로컬 개발용, streamable-http는 원격 MCP 서버용입니다. 전송 방식은 서버가 어디에서 어떻게 떠 있는지에 따라 달라집니다. 이 코드는 MCP로 외부 도구를 표준 인터페이스에 연결하는 방식을 다룹니다. 도구 구현과 에이전트 사용 지점을 분리해서 봅니다. 화면에서 stdio는 로컬 서브프로세스로 서버를 실행해 표준 입출력으로 통신합니다. streamable-http는 원격 서버

URL과 인증 헤더를 가진 웹 기반 연결입니다. 인증은 정적 Bearer 헤더, 커스텀 httpx2.Auth, MCP OAuth2 흐름을 상황별로 씁니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 sys; os를 준비합니다. 두 코드 블록을 비교하면 로컬 서버 블록은 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 원격 서버 블록은 이 코드는 MCP로 외부 도구를 표준 인터페이스에 연결하는 방식을 다룹니다. 도구 구현과 에이전트 사용 지점을 분리해서 봅니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

82. MCP 도구는 MCPAdapter 컨텍스트에서 list_tools()로 가져옵니다

MCP 도구 discovery는 MCPAdapter 컨텍스트 안에서 처리합니다. async with 블록으로 연결을 열고, list_tools로 도구를 가져온 뒤 create_agent에 전달합니다. 반환된 도구는 client context를 보존하므로 agent 호출 함수 바깥으로 넘길 수 있습니다. 기존 get_tools 중심 예제와 달리 discovery와 호출 범위가 코드에 드러납니다.

---

83. 원격 MCP 인증은 FastMCP Client나 transport에서 관리합니다

원격 MCP 서버 인증은 FastMCP Client가 소유합니다. 서버 설정 딕셔너리에 임의 auth 객체를 넣지 않고, Client에 Bearer 토큰이나 oauth 설정을 둔 다음 MCPAdapter에 넘깁니다. 여러 서버를 묶을 때는 ClientGroup을 사용해 서버별 인증 경계를 유지합니다.

---

84. FastMCP 데코레이터 하나로 직접 MCP 서버를 만듭니다

FastMCP 서버는 도구와 리소스를 노출하는 쪽입니다. 에이전트는 이 서버를 stdio 설정으로 등록한 뒤 MCPAdapter 컨텍스트에서 list_tools를 호출합니다. 서버 구현과 에이전트 사용 지점을 분리해 읽으면 됩니다.

---

85. MCP 운영은 Elicitation과 LangGraph interrupt까지 함께 봐야 합니다

MCP 운영에서는 InputRequiredResult 기반 Elicitation을 기준으로 설명합니다. 현대 MCP 서버가 추가 입력을 요구하면 LangGraph interrupt로 노출되고, payload.requests[0].key를 읽어 responses의 같은 key 아래에 accept, decline, cancel answer object를 넣어 재개합니다. legacy ctx.elicit 푸시 핸드셰이크는 이 수업의 재개 계약으로 설명하지 않습니다.

---

86. 프론트엔드 스트리밍

이제 PART 13 프론트엔드 스트리밍으로 넘어갑니다. 백엔드가 내보내는 여러 층의 스트림을 프런트엔드가 어떻게 받아서 채팅 UI로 보여주는지 구분합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/12_frontend_streaming.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

---

87. 스트리밍 API는 저수준 토큰에서 UI projection까지 층이 나뉩니다

스트리밍 API는 저수준 토큰에서 UI projection까지 층이 나뉩니다. UI가 필요한 것은 토큰뿐 아니라 도구 타임라인, 상태 스냅샷, 최종 출력입니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 .stream()는 토큰 또는 stream_mode별 청크입니다. 판단 기준은 간단한 콘솔·채팅 토큰 출력입니다. .astream_events()는 on_chat_model_start/end, on_tool_start/end입니다. 판단 기준은 세밀한 이벤트 로그·디버그 타임라인입니다. stream_events(version='v3')는 messages, tool_calls, values, output projection입니다. 판단 기준은 Agent UI의 표준 데이터 모델입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

88. astream_events(version="v2")는 모델 호출의 시작·진행·종료를 구분합니다

astream_events(version="v2")는 모델 호출의 시작·진행·종료를 구분합니다. 토큰만 필요하면 stream()으로 충분하지만, 모델 호출의 생애주기를 세밀히 추적할 때는 내부 이벤트를 그대로 받습니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 화면에서 event 필드는 event['event'] 문자열로 종류를 구분합니다. on_chat_model_start / on_chat_model_stream / on_chat_model_end입니다. 토큰 추출은 on_chat_model_stream일 때만 event['data']['chunk'].content에 실제 토큰이 담깁니다. 비동기 전용은 async for 구조라 동기 stream()과 달리 async 함수 안에서만 소비합니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 astream_events입니다. 노트북 기준 위치는 12_frontend_streaming.ipynb astream_events v2입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

89. Event Streaming v3는 메시지·도구·상태를 projection으로 분리합니다

Event Streaming v3는 메시지·도구·상태를 projection으로 분리합니다. LangChain 1.3+에서는 agent 실행 UI에 stream_events(version='v3')를 우선 검토합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 messages는 채팅 버블로 렌더링할 모델 메시지와 token delta입니다. tool_calls는 도구 시작·출력·완료·오류를 타임라인으로 다룹니다. output는 최종 state를 저장하거나 후처리합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 함수 get_weather는 실제 작업 단위입니다. 핵심 API는 create_agent, stream입니다. 노트북 기준 위치는 12_frontend_streaming.ipynb Event Streaming v3입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

90. useStream은 스레드·재연결·중단·커스텀 이벤트를 한 훅으로 묶습니다

useStream은 스레드·재연결·중단·커스텀 이벤트를 한 훅으로 묶습니다. React UI는 서버 스트림을 직접 파싱하기보다 훅의 상태와 콜백으로 렌더링합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 assistantId / apiUrl은 어떤 배포 그래프와 통신할지 지정입니다. 판단 기준은 로컬은 127.0.0.1:2024, 배포는 Deployment URL입니다. threadId / onThreadId는 대화를 이어갈 스레드 관리입니다. 판단 기준은 URL 파라미터나 localStorage에 보관 가능입니다. reconnectOnMount는 새로고침 뒤 진행 중 스트림 재연결입니다. 판단 기준은 긴 실행 작업에서 사용자 경험 유지입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

91. useStream 훅 하나로 메시지·로딩·에러 상태를 함께 관리합니다

useStream 훅 하나로 메시지·로딩·에러 상태를 함께 관리합니다. @langchain/langgraph-sdk/react의 useStream은 서버와의 스트리밍 연결, 메시지 누적, 로딩·에러 상태를 캡슐화합니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 화면에서 필수 연결 정보는 assistantId와 apiUrl만 주면 즉시 연결됩니다. submit()으로 전송은 stream.submit({ messages: [...] }) 호출 한 번으로 사용자 메시지를 보냅니다. 자동 상태 관리는 stream.messages, stream.isLoading, stream.error를 그대로 렌더링하면 됩니다. 코드는 다음 지점을 중심으로 읽습니다. 핵심 API는 stream입니다. 노트북 기준 위치는 Chat.tsx useStream 실사용입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

92. 브랜칭과 멀티에이전트 UI는 checkpoint와 langgraph_node 메타데이터가 핵심입니다

브랜칭과 멀티에이전트 UI는 checkpoint와 langgraph_node 메타데이터가 핵심입니다. 사용자가 메시지를 고쳐 다른 경로를 만들거나, 여러 에이전트의 발화를 구분하려면 스트림 메타를 보존해야 합니다. 묶음별로 읽으면 브랜칭에서는 메시지 메타데이터의 parent_checkpoint를 submit 옵션에 넣으면 해당 시점에서 대체 경로를 만듭니다; 사용자 메시지 편집과 AI 응답 재생성 모두 같은 checkpoint 패턴을 씁니다를 봅니다. 멀티에이전트 스트리밍에서는 메타데이터의 langgraph_node로 researcher, writer, reviewer 같은 발화 출처를 구분합니다; 노드별 라벨과 색상을 매핑하면 사용자가 어떤 에이전트가 말하는지 즉시 알 수 있습니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

93. 가드레일

이제 PART 14 가드레일으로 넘어갑니다. PII, HITL, before_agent, after_agent, 다층 방어를 마지막 운영 경계로 정리합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/13_guardrails.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다.

94. 가드레일은 결정론적 규칙과 모델 기반 검사를 조합합니다

가드레일은 결정론적 규칙과 모델 기반 검사를 조합합니다. 빠른 실패는 비용을 줄이고, 모델 기반 검사는 미묘한 위반을 보완합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 결정론적은 정규식, 키워드, 명시 규칙입니다. 판단 기준은 빠르고 예측 가능하지만 미묘한 의미 위반을 놓칠 수 있습니다. 모델 기반은 LLM·분류기로 의미 분석입니다. 판단 기준은 정교하지만 느리고 비용이 높습니다. 입력 가드레일은 before_agent에서 요청 검증입니다. 판단 기준은 모델 호출 전 차단해 위험한 컨텍스트 유입을 막습니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

95. PIIMiddleware는 개인정보 유형별로 redact·mask·hash·block을 선택합니다

PIIMiddleware는 개인정보 유형별로 redact·mask·hash·block을 선택합니다. 입력, 출력, 도구 결과 중 어느 표면에 적용할지 apply_to_*를 명시해야 동작합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 redact는 값을 [REDACTED_EMAIL]처럼 대체합니다. mask는 신용카드 마지막 4자리처럼 일부만 남깁니다. hash는 값을 결정론적 해시로 대체해 원본 없이도 동일성 비교가 가능합니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 PIIMiddleware를 준비합니다. 핵심 API는 create_agent, PIIMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 13_guardrails.ipynb PII guardrail입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

96. 가드레일은 여러 미들웨어를 쌓아 다층 방어로 조합합니다

가드레일은 여러 미들웨어를 쌓아 다층 방어로 조합합니다. 빠른 결정론적 체크를 앞에, 느린 모델 기반 체크를 뒤에 배치해야 비용과 지연을 줄입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 화면에서 Layer 1~2은 키워드 필터와 PII 마스킹처럼 규칙 기반 가드레일을 입력 단계에 먼저 둡니다. Layer 3은 이메일 전송처럼 민감한 도구 호출에는 HumanInTheLoopMiddleware로 사람 승인을 끼워 넣습니다. Layer 4은 모델 기반 안전성 검사는 가장 비싸므로 마지막 단계에 배치합니다(노트북에서 커스텀으로 정의). 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 PIIMiddleware, HumanInTheLoopMiddleware를 준비합니다. 핵심 API는 create_agent,

HumanInTheLoopMiddleware, PIIMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 노트북 기준 위치는 13_guardrails.ipynb 다중 가드레일 조합 (다층 방어)입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

97. 커스텀 입력 가드레일은 before_agent에서 jump_to='end'로 안전하게 종료합니다

커스텀 입력 가드레일은 before_agent에서 jump_to='end'로 안전하게 종료합니다. 금지 키워드를 발견했을 때 예외만 던지기보다 사용자에게 거절 메시지를 남기는 패턴이 운영에 적합합니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 화면에서 before_agent는 에이전트 실행 시작 전 요청을 검사합니다. can_jump_to는 이 훅이 종료 노드로 점프 권한을 선언합니다. graceful refusal은 사용자는 빈 에러가 아니라 정책 안내를 받습니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 before_agent, AgentState, hook_config; runtime에서 Runtime를 준비합니다. 데코레이터 before_agent가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 content_filter는 실제 작업 단위입니다. 노트북 기준 위치는 13_guardrails.ipynb before_agent guardrail입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

98. 출력 가드레일은 after_agent에서 최종 응답을 검증하거나 수정합니다

출력 가드레일은 after_agent에서 최종 응답을 검증하거나 수정합니다. 모델 기반 안전성 검사는 느리므로 결정론적 입력 차단 뒤 마지막 방어선으로 배치합니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 화면에서 after_agent는 에이전트 실행 후 마지막 메시지를 검사합니다. safety_model은 가벼운 모델로 SAFE/UNSAFE 같은 판정을 받습니다. 다층 방어는 ContentFilter → PII → HITL → Safety 순서처럼 빠른 검사를 앞에 둡니다. 코드는 다음 지점을 중심으로 읽습니다. 먼저 import는 middleware에서 after_agent, AgentState, hook_config; chat_models에서 init_chat_model를 준비합니다. 데코레이터 after_agent가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 safety_check는 실제 작업 단위입니다. 노트북 기준 위치는 13_guardrails.ipynb after_agent guardrail입니다. 강의에서는 이 코드를 한 줄씩 타이핑하기보다, 객체가 만들어지는 순서와 다음 호출로 전달되는 값을 중심으로 설명하면 됩니다.

---

99. 프로덕션 가드레일은 빠른 실패와 친절한 거절을 함께 설계해야 합니다

프로덕션 가드레일은 빠른 실패와 친절한 거절을 함께 설계해야 합니다. 안전 장치 자체도 테스트·로깅·폴백 대상입니다. 묶음별로 읽으면 다층 방어에서는 결정론적 입력 필터를 가장 앞에 두어 불필요한 모델 호출을 줄입니다; PII 미들웨어는 입력·출력·도구 결과 중 필요한 표면에만 명시적으로 적용합니다; 민감 도구는 HumanInTheLoopMiddleware로 승인 흐름을 둡니다를 봅니다. 운영 품질에서는 차단 이벤트는 LangSmith trace와 감사 로그에 남깁니다; 가드레일이 실패할 때의 기본 정책을 try/except와 폴백으로 정합니다; 도메인별 테스트 케이스를 GenericFakeChatModel과 함께 CI에 넣습니다를 봅니다. 이런 정리 슬라이드는 실습 중 체크리스트로 사용하면 좋습니다. 코드가 동작하는지뿐 아니라, 상태와 권한과 실패 처리를 함께 확인해야 합니다.

---

100. 개념 근거와 LangChain v1 모델

이제 REFERENCE 개념 근거와 LangChain v1 모델으로 넘어갑니다. LangChain v1 개념 설명은 공식 문서, ReAct 논문, MCP 공식 문서를 기준으로 보강합니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 여기서는 개념 설명의 근거를 정리합니다. 실무에서 프레임워크를 설명할 때는 개인적인 비유보다 공식 문서와 논문에 기대는 편이 안전합니다.

101. create_agent()는 메시지, 모델 호출, 도구 호출을 반복하는 실행 루프입니다

create_agent()는 메시지, 모델 호출, 도구 호출을 반복하는 실행 루프입니다. LangChain 공식 문서는 create_agent를 모델·도구·시스템 프롬프트·미들웨어를 묶는 agent harness로 설명합니다. ReAct 논문은 reasoning과 acting의 교차 루프를 배경으로 제공합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Messages는 System, Human, AI, Tool 메시지가 실행 문맥을 이룹니다. Model은 모델은 다음 답변 또는 tool call을 생성합니다. Tool result는 도구 결과는 ToolMessage로 돌아와 다음 모델 입력이 됩니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

102. 미들웨어는 agent loop 안의 고정 지점과 호출 주변을 제어합니다

미들웨어는 agent loop 안의 고정 지점과 호출 주변을 제어합니다. LangChain 공식 문서는 node-style hook과 wrap-style hook을 구분합니다. 로깅, 동적 프롬프트, 재시도, 가드레일, HITL 같은 운영 정책을 여기에 둡니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Node-style는 before_agent, before_model, after_model, after_agent에서 state를 갱신합니다. Wrap-style는 wrap_model_call, wrap_tool_call로 각 호출을 감쌉니다. Policy는 정책은 앱 바깥이 아니라 agent 실행 흐름 안에서 적용됩니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

103. MCP는 에이전트와 외부 도구 사이의 연결 방식을 표준화합니다

MCP는 에이전트와 외부 도구 사이의 연결 방식을 표준화합니다. MCP 공식 문서는 AI 애플리케이션이 데이터 소스, 도구, 프롬프트에 표준 방식으로 연결되도록 하는 프로토콜로 설명합니다. LangChain은 MCP client에서 가져온 도구를 create_agent에 연결합니다. 그림은 코드보다 먼저 봐야 할 실행 구조를 압축합니다. 화면에서 Client는 서버별 세션과 인증을 관리하고 도구 목록을 가져옵니다. Server은 도구, 리소스, 프롬프트를 표준 인터페이스로 노출합니다. Transport는 로컬은 stdio, 원격은 streamable-http를 주로 사용합니다. 이 슬라이드는 이름 암기가 아니라, 다음 코드에서 어느 객체가 어떤 역할을 맡는지 미리 잡기 위한 장면입니다.

104. LangChain 개념 보강 출처는 공식 문서와 논문을 기준으로 삼습니다

LangChain 개념 보강 출처는 공식 문서와 논문을 기준으로 삼습니다. 덱의 개념 설명은 노트북 코드와 아래 신뢰 출처를 함께 반영합니다. 이 표는 세부 기능 암기용이 아니라, 상황에 맞는 선택 기준을 잡기 위한 자료입니다. 대표 행을 읽어보면 Agent loop는 LangChain Agents official docs, ReAct paper입니다. 판단 기준은 create_agent, tool call, ToolMessage 루프 설명입니다. Messages/Tools는 LangChain Messages and Tools official docs입니다. 판단 기준은 메시지 타입, 도구 스키마, 도구 결과 설명입니다. Structured output는 LangChain Structured Output official docs입니다. 판단 기준은 response_format과 structured_response 설명입니다. 강의에서는 모든 칸을 같은 비중으로 읽지 않습니다. 왼쪽의 개념 이름과 오른쪽의 판단 기준을 연결해 설명합니다.

105. 01_introduction.ipynb

이제 CODE 01_introduction.ipynb로 넘어갑니다. 01_introduction.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/01_introduction.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

106. 01_introduction 코드 01

이제 01_introduction 코드 01 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 subprocess; importlib를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

107. 01_introduction 코드 02

이제 01_introduction 코드 02 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

108. 01_introduction 코드 03

이제 01_introduction 코드 03 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

109. 01_introduction 코드 04

이제 01_introduction 코드 04 슬라이드입니다. 노트북 위치는 01_introduction.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; tools에서 tool를 준비합니다. 핵심 API는 create_agent, ChatOpenAI, InMemorySaver입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

110. 02_quickstart.ipynb

이제 CODE 02_quickstart.ipynb로 넘어갑니다. 02_quickstart.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/02_quickstart.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

111. 02_quickstart 코드 01

이제 02_quickstart 코드 01 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

112. 02_quickstart 코드 02

이제 02_quickstart 코드 02 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 6이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

113. 02_quickstart 코드 03

이제 02_quickstart 코드 03 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

114. 02_quickstart 코드 04

이제 02_quickstart 코드 04 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 add, multiply는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

115. 02_quickstart 코드 05

이제 02_quickstart 코드 05 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

116. 02_quickstart 코드 06

이제 02_quickstart 코드 06 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드

에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

117. 02_quickstart 코드 07

이제 02_quickstart 코드 07 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

118. 02_quickstart 코드 08

이제 02_quickstart 코드 08 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

119. 02_quickstart 코드 09 (1/2)

이제 02_quickstart 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 memory에서 InMemorySaver를 준비합니다. 핵심 API는 create_agent, InMemorySaver, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

120. 02_quickstart 코드 09 (2/2)

이제 02_quickstart 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

121. 02_quickstart 코드 10

이제 02_quickstart 코드 10 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

122. 02_quickstart 코드 11 (1/2)

이제 02_quickstart 코드 11 (1/2) 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 dataclasses에서 dataclass; agents에서 create_agent를 준비합니다. 클래스 Context는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 핵심 API는 create_agent, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

123. 02_quickstart 코드 11 (2/2)

이제 02_quickstart 코드 11 (2/2) 슬라이드입니다. 노트북 위치는 02_quickstart.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

124. 03_models_and_messages.ipynb

이제 CODE 03_models_and_messages.ipynb로 넘어갑니다. 03_models_and_messages.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/03_models_and_messages.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

125. 03_models_and_messages 코드 01

이제 03_models_and_messages 코드 01 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

126. 03_models_and_messages 코드 02

이제 03_models_and_messages 코드 02 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

127. 03_models_and_messages 코드 03

이제 03_models_and_messages 코드 03 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 chat_models에서 init_chat_model를 준비합니다. 핵심 API는 init_chat_model, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

128. 03_models_and_messages 코드 04

이제 03_models_and_messages 코드 04 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

129. 03_models_and_messages 코드 05

이제 03_models_and_messages 코드 05 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

130. 03_models_and_messages 코드 06

이제 03_models_and_messages 코드 06 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

131. 03_models_and_messages 코드 07

이제 03_models_and_messages 코드 07 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 messages에서 SystemMessage, HumanMessage, AIMessage를 준비합니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

132. 03_models_and_messages 코드 08

이제 03_models_and_messages 코드 08 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

133. 03_models_and_messages 코드 09 (1/2)

이제 03_models_and_messages 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 messages에서 HumanMessage; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI, invoke입니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

134. 03_models_and_messages 코드 09 (2/2)

이제 03_models_and_messages 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

135. 03_models_and_messages 코드 10

이제 03_models_and_messages 코드 10 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 callbacks에서 UsageMetadataCallbackHandler를 준비합니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

136. 03_models_and_messages 코드 11

이제 03_models_and_messages 코드 11 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

137. 03_models_and_messages 코드 12

이제 03_models_and_messages 코드 12 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 chat_models에서 init_chat_model를 준비합니다. 핵심 API는 init_chat_model, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

138. 03_models_and_messages 코드 13

이제 03_models_and_messages 코드 13 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 25이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI; os를 준비합니다. 핵심 API는 ChatOpenAI, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

139. 03_models_and_messages 코드 14 (1/2)

이제 03_models_and_messages 코드 14 (1/2) 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

140. 03_models_and_messages 코드 14 (2/2)

이제 03_models_and_messages 코드 14 (2/2) 슬라이드입니다. 노트북 위치는 03_models_and_messages.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

141. 04_tools_and_structured_output.ipynb

이제 CODE 04_tools_and_structured_output.ipynb로 넘어갑니다. 04_tools_and_structured_output.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/04_tools_and_structured_output.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

142. 04_tools_and_structured_output 코드 01

이제 04_tools_and_structured_output 코드 01 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

143. 04_tools_and_structured_output 코드 02

이제 04_tools_and_structured_output 코드 02 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

144. 04_tools_and_structured_output 코드 03

이제 04_tools_and_structured_output 코드 03 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_weather는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

145. 04_tools_and_structured_output 코드 04

이제 04_tools_and_structured_output 코드 04 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 pydantic에서 BaseModel, Field를 준비합니다. 클래스 SearchQuery는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 search_database는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

146. 04_tools_and_structured_output 코드 05

이제 04_tools_and_structured_output 코드 05 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

147. 04_tools_and_structured_output 코드 06

이제 04_tools_and_structured_output 코드 06 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool, ToolRuntime를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_user_info는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

148. 04_tools_and_structured_output 코드 07 (1/2)

이제 04_tools_and_structured_output 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool, ToolRuntime; memory에서 InMemorySaver를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 debug_trace는 실제 작업 단위입니다. 핵심 API는 create_agent, InMemorySaver입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

149. 04_tools_and_structured_output 코드 07 (2/2)

이제 04_tools_and_structured_output 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

150. 04_tools_and_structured_output 코드 08 (1/2)

이제 04_tools_and_structured_output 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 who_am_i는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

151. 04_tools_and_structured_output 코드 08 (2/2)

이제 04_tools_and_structured_output 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹

니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

152. 04_tools_and_structured_output 코드 09 (1/2)

이제 04_tools_and_structured_output 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 messages에서 ToolMessage를 준비합니다. 데코레이터 wrap_tool_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 safe_executor는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

153. 04_tools_and_structured_output 코드 09 (2/2)

이제 04_tools_and_structured_output 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

154. 04_tools_and_structured_output 코드 10 (1/2)

이제 04_tools_and_structured_output 코드 10 (1/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command; messages에서 ToolMessage를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 remember_user_pref는 실제 작업 단위입니다. 핵심 API는 create_agent, Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

155. 04_tools_and_structured_output 코드 10 (2/2)

이제 04_tools_and_structured_output 코드 10 (2/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

156. 04_tools_and_structured_output 코드 11

이제 04_tools_and_structured_output 코드 11 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 pydantic에서 BaseModel를 준비합니다. 클래스 MovieReview는 데이터 구조나 정책 단위를 정의합니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

157. 04_tools_and_structured_output 코드 12 (1/2)

이제 04_tools_and_structured_output 코드 12 (1/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 25이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 structured_output에서 ToolStrategy; dataclasses에서 dataclass를 준비합니다. 클래스 CalculationResult는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 calculate는 실제 작업 단위입니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

158. 04_tools_and_structured_output 코드 12 (2/2)

이제 04_tools_and_structured_output 코드 12 (2/2) 슬라이드입니다. 노트북 위치는 04_tools_and_structured_output.ipynb의 cell 25이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 ToolStrategy, invoke입니다. 설정값 response_format는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

159. 05_memory_and_streaming.ipynb

이제 CODE 05_memory_and_streaming.ipynb로 넘어갑니다. 05_memory_and_streaming.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/05_memory_and_streaming.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

160. 05_memory_and_streaming 코드 01

이제 05_memory_and_streaming 코드 01 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서

load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

161. 05_memory_and_streaming 코드 02

이제 05_memory_and_streaming 코드 02 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

162. 05_memory_and_streaming 코드 03 (1/2)

이제 05_memory_and_streaming 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_time는 실제 작업 단위입니다. 핵심 API는 create_agent, InMemorySaver, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

163. 05_memory_and_streaming 코드 03 (2/2)

이제 05_memory_and_streaming 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

164. 05_memory_and_streaming 코드 04

이제 05_memory_and_streaming 코드 04 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

165. 05_memory_and_streaming 코드 05 (1/2)

이제 05_memory_and_streaming 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는

messages에서 trim_messages; middleware에서 before_model를 준비합니다. 데코레이터 before_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 trim_context는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

166. 05_memory_and_streaming 코드 05 (2/2)

이제 05_memory_and_streaming 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 InMemorySaver입니다. 설정값 context, middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

167. 05_memory_and_streaming 코드 06 (1/3)

이제 05_memory_and_streaming 코드 06 (1/3) 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 dataclasses에서 dataclass; memory에서 InMemoryStore를 준비합니다. 클래스 Context는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 save_preference는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

168. 05_memory_and_streaming 코드 06 (2/3)

이제 05_memory_and_streaming 코드 06 (2/3) 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_preference는 실제 작업 단위입니다. 핵심 API는 create_agent, InMemorySaver입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

169. 05_memory_and_streaming 코드 06 (3/3)

이제 05_memory_and_streaming 코드 06 (3/3) 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable, context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

170. 05_memory_and_streaming 코드 07

이제 05_memory_and_streaming 코드 07 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

171. 05_memory_and_streaming 코드 08

이제 05_memory_and_streaming 코드 08 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

172. 05_memory_and_streaming 코드 09

이제 05_memory_and_streaming 코드 09 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

173. 05_memory_and_streaming 코드 10

이제 05_memory_and_streaming 코드 10 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

174. 05_memory_and_streaming 코드 11

이제 05_memory_and_streaming 코드 11 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

175. 05_memory_and_streaming 코드 12

이제 05_memory_and_streaming 코드 12 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

176. 05_memory_and_streaming 코드 13

이제 05_memory_and_streaming 코드 13 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 25이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

177. 05_memory_and_streaming 코드 14

이제 05_memory_and_streaming 코드 14 슬라이드입니다. 노트북 위치는 05_memory_and_streaming.ipynb의 cell 27이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 stream, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

178. 06_middleware.ipynb

이제 CODE 06_middleware.ipynb로 넘어갑니다. 06_middleware.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/06_middleware.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

179. 06_middleware 코드 01

이제 06_middleware 코드 01 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

180. 06_middleware 코드 02

이제 06_middleware 코드 02 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

181. 06_middleware 코드 03 (1/2)

이제 06_middleware 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; tools에서 tool를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 search는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

182. 06_middleware 코드 03 (2/2)

이제 06_middleware 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

183. 06_middleware 코드 04

이제 06_middleware 코드 04 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

184. 06_middleware 코드 05

이제 06_middleware 코드 05 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 ModelFallbackMiddleware를 준비합니다. 핵심 API는 create_agent, ChatOpenAI입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

185. 06_middleware 코드 06

이제 06_middleware 코드 06 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 ContextEditingMiddleware, ClearToolUsesEdit를 준비합니다. 핵심 API는 create_agent입니다. 설정값 context, middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

186. 06_middleware 코드 07

이제 06_middleware 코드 07 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 before_model를 준비합니다. 데코레이터 before_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 log_model_input는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

187. 06_middleware 코드 08 (1/2)

이제 06_middleware 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 after_model를 준비합니다. 데코레이터 after_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 log_model_output는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

188. 06_middleware 코드 08 (2/2)

이제 06_middleware 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

189. 06_middleware 코드 09 (1/2)

이제 06_middleware 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 time를 준비합니다. 데코레이터 wrap_model_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 retry_on_error는 실제 작업 단위입니다. 설정값 response_format, middleware는 실행 결과보다 운영 경계를 설명합니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

190. 06_middleware 코드 09 (2/2)

이제 06_middleware 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

191. 06_middleware 코드 10

이제 06_middleware 코드 10 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 20이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 wrap_model_call, ModelRequest, ModelResponse를 준비합니다. 데코레이터 wrap_model_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 restrict_tools_for_anonymous는 실제 작업 단위입니다. 설정값 context, middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

192. 06_middleware 코드 11

이제 06_middleware 코드 11 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 dynamic_prompt; datetime에서 datetime를 준비합니다. 데코레이터 dynamic_prompt가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 add_datetime_context는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

193. 06_middleware 코드 12 (1/2)

이제 06_middleware 코드 12 (1/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 wrap_tool_call; time를 준비합니다. 데코레이터 wrap_tool_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 tool_timing_logger는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

194. 06_middleware 코드 12 (2/2)

이제 06_middleware 코드 12 (2/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 24이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니

다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

195. 06_middleware 코드 13 (1/2)

이제 06_middleware 코드 13 (1/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 before_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 keyword_guardrail는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

196. 06_middleware 코드 13 (2/2)

이제 06_middleware 코드 13 (2/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

197. 06_middleware 코드 14 (1/2)

이제 06_middleware 코드 14 (1/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 before_model, hook_config; messages에서 AIMessage를 준비합니다. 데코레이터 hook_config, before_model가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 jumping_guardrail는 실제 작업 단위입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

198. 06_middleware 코드 14 (2/2)

이제 06_middleware 코드 14 (2/2) 슬라이드입니다. 노트북 위치는 06_middleware.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

199. 07_hitl_and_runtime.ipynb

이제 CODE 07_hitl_and_runtime.ipynb로 넘어갑니다. 07_hitl_and_runtime.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/07_hitl_and_runtime.ipynb입니다. 슬라이

드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

200. 07_hitl_and_runtime 코드 01

이제 07_hitl_and_runtime 코드 01 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

201. 07_hitl_and_runtime 코드 02

이제 07_hitl_and_runtime 코드 02 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

202. 07_hitl_and_runtime 코드 03 (1/2)

이제 07_hitl_and_runtime 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; tools에서 tool를 준비합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 send_email, delete_file는 실제 작업 단위입니다. 핵심 API는 create_agent, InMemorySaver, HumanInTheLoopMiddleware입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

203. 07_hitl_and_runtime 코드 03 (2/2)

이제 07_hitl_and_runtime 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, InMemorySaver입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

204. 07_hitl_and_runtime 코드 04 (1/2)

이제 07_hitl_and_runtime 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니

다. 먼저 import는 types에서 Command를 준비합니다. 핵심 API는 invoke, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

205. 07_hitl_and_runtime 코드 04 (2/2)

이제 07_hitl_and_runtime 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 Command입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

206. 07_hitl_and_runtime 코드 05 (1/2)

이제 07_hitl_and_runtime 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 ToolRuntime; dataclasses에서 dataclass를 준비합니다. 클래스 UserContext는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 dataclass, tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_user_profile, check_permissions는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

207. 07_hitl_and_runtime 코드 05 (2/2)

이제 07_hitl_and_runtime 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

208. 07_hitl_and_runtime 코드 06 (1/2)

이제 07_hitl_and_runtime 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 whoami_exec는 실제 작업 단위입니다. 핵심 API는 create_agent, InMemorySaver, invoke입니다. 설정값 thread_id, context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

209. 07_hitl_and_runtime 코드 06 (2/2)

이제 07_hitl_and_runtime 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

210. 07_hitl_and_runtime 코드 07 (1/2)

이제 07_hitl_and_runtime 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 whoami_server는 실제 작업 단위입니다. 핵심 API는 create_agent, InMemorySaver, invoke입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

211. 07_hitl_and_runtime 코드 07 (2/2)

이제 07_hitl_and_runtime 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 설정값 thread_id, configurable, context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

212. 07_hitl_and_runtime 코드 08 (1/2)

이제 07_hitl_and_runtime 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 dynamic_prompt를 준비합니다. 데코레이터 tool, tool, dynamic_prompt가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 basic_search, advanced_analytics, role_based_prompt는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

213. 07_hitl_and_runtime 코드 08 (2/2)

이제 07_hitl_and_runtime 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 07_hitl_and_runtime.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

214. 07_hitl_and_runtime 코드 09

이 HITL 런타임 부록 코드는 현재 MCPAdapter 방식으로 읽습니다. 로컬 FastMCP echo 서버를 만들고, MCPConfig를 adapter target으로 넘긴 뒤 list_tools 결과를 create_agent에 연결합니다. 예전 langchain.mcp 세션 헬퍼는 여기서 사용하지 않습니다.

215. 08_multi_agent.ipynb

이제 CODE 08_multi_agent.ipynb로 넘어갑니다. 08_multi_agent.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/08_multi_agent.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

216. 08_multi_agent 코드 01

이제 08_multi_agent 코드 01 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

217. 08_multi_agent 코드 02

이제 08_multi_agent 코드 02 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

218. 08_multi_agent 코드 03 (1/2)

이제 08_multi_agent 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; tools에서 tool를 준비합니다. 데코레이터 tool, tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 math_expert, code_expert, general_search는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

219. 08_multi_agent 코드 03 (2/2)

이제 08_multi_agent 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니

다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

220. 08_multi_agent 코드 04 (1/2)

이제 08_multi_agent 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; tools에서 tool를 준비합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 calc_step는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

221. 08_multi_agent 코드 04 (2/2)

이제 08_multi_agent 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 함수 math_expert_v2는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

222. 08_multi_agent 코드 05 (1/3)

이제 08_multi_agent 코드 05 (1/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 typing에서 Literal를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 dispatch는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

223. 08_multi_agent 코드 05 (2/3)

이제 08_multi_agent 코드 05 (2/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 list_subagents, describe_subagent는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

224. 08_multi_agent 코드 05 (3/3)

이제 08_multi_agent 코드 05 (3/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 13이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

225. 08_multi_agent 코드 06 (1/2)

이제 08_multi_agent 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool, ToolRuntime; agents에서 AgentState를 준비합니다. 클래스 SupportState는 데이터 구조나 정책 단위를 정의합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 lookup_account는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

226. 08_multi_agent 코드 06 (2/2)

이제 08_multi_agent 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

227. 08_multi_agent 코드 07 (1/3)

이제 08_multi_agent 코드 07 (1/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 threading; uuid를 준비합니다. 함수 _background_worker는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

228. 08_multi_agent 코드 07 (2/3)

이제 08_multi_agent 코드 07 (2/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool, tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 start_job, check_status, get_result는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

229. 08_multi_agent 코드 07 (3/3)

이제 08_multi_agent 코드 07 (3/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

230. 08_multi_agent 코드 08 (1/2)

이제 08_multi_agent 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command; messages에서 ToolMessage를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 record_finding는 실제 작업 단위입니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

231. 08_multi_agent 코드 08 (2/2)

이제 08_multi_agent 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent, invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

232. 08_multi_agent 코드 09 (1/3)

이제 08_multi_agent 코드 09 (1/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Command; graph에서 StateGraph, START, END를 준비합니다. 데코레이터 tool, tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 transfer_to_sales, transfer_to_support, resolve_query는 실제 작업 단위입니다. 핵심 API는 create_agent, StateGraph, Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

233. 08_multi_agent 코드 09 (2/3)

이제 08_multi_agent 코드 09 (2/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

234. 08_multi_agent 코드 09 (3/3)

이제 08_multi_agent 코드 09 (3/3) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

235. 08_multi_agent 코드 10 (1/5)

이제 08_multi_agent 코드 10 (1/5) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 5장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 middleware에서 wrap_model_call; agents에서 AgentState를 준비합니다. 클래스 HandoffState는 데이터 구조나 정책 단위를 정의합니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

236. 08_multi_agent 코드 10 (2/5)

이제 08_multi_agent 코드 10 (2/5) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 transfer_to_sales_v2, transfer_to_support_v2는 실제 작업 단위입니다. 핵심 API는 Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

237. 08_multi_agent 코드 10 (3/5)

이제 08_multi_agent 코드 10 (3/5) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 resolve_query_v2는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

238. 08_multi_agent 코드 10 (4/5)

이제 08_multi_agent 코드 10 (4/5) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 wrap_model_call가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 persona_router는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

239. 08_multi_agent 코드 10 (5/5)

이제 08_multi_agent 코드 10 (5/5) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 5번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

240. 08_multi_agent 코드 11 (1/2)

이제 08_multi_agent 코드 11 (1/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 load_skill는 실제 작업 단위입니다. 핵심 API는 create_agent, invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

241. 08_multi_agent 코드 11 (2/2)

이제 08_multi_agent 코드 11 (2/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 26이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

242. 08_multi_agent 코드 12 (1/2)

이제 08_multi_agent 코드 12 (1/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 types에서 Send를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 classify_query는 실제 작업 단위입니다. 핵심 API는 create_agent, Send입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

243. 08_multi_agent 코드 12 (2/2)

이제 08_multi_agent 코드 12 (2/2) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 28이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

244. 08_multi_agent 코드 13 (1/4)

이제 08_multi_agent 코드 13 (1/4) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 4장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; types에서 Command, Send를 준비합니다. 클래스 FanOutState는 데이터 구조나 정책 단위를 정의합니다. 함수 classifier는 실제 작업 단위입니다. 핵심 API는 StateGraph, Command, Send입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

245. 08_multi_agent 코드 13 (2/4)

이제 08_multi_agent 코드 13 (2/4) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 math_node, code_node, general_node, aggregator는 실제 작업 단위입니다. 핵심 API는 Command, Send입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

246. 08_multi_agent 코드 13 (3/4)

이제 08_multi_agent 코드 13 (3/4) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge, Command입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

247. 08_multi_agent 코드 13 (4/4)

이제 08_multi_agent 코드 13 (4/4) 슬라이드입니다. 노트북 위치는 08_multi_agent.ipynb의 cell 30이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 4번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

248. 09_custom_workflow_and_rag.ipynb

이제 CODE 09_custom_workflow_and_rag.ipynb로 넘어갑니다. 09_custom_workflow_and_rag.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/09_custom_workflow_and_rag.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

249. 09_custom_workflow_and_rag 코드 01

이제 09_custom_workflow_and_rag 코드 01 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 create_agent, ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

250. 09_custom_workflow_and_rag 코드 02

이제 09_custom_workflow_and_rag 코드 02 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

251. 09_custom_workflow_and_rag 코드 03 (1/3)

이제 09_custom_workflow_and_rag 코드 03 (1/3) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 StateGraph, START, END; typing에서 TypedDict를 준비합니다. 클래스 WorkflowState는 데이터 구조나 정책 단위를 정의합니다. 함수 preprocess, analyze는 실제 작업 단위입니다. 핵심 API는 StateGraph입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

252. 09_custom_workflow_and_rag 코드 03 (2/3)

이제 09_custom_workflow_and_rag 코드 03 (2/3) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, add_node, add_edge입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

253. 09_custom_workflow_and_rag 코드 03 (3/3)

이제 09_custom_workflow_and_rag 코드 03 (3/3) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

254. 09_custom_workflow_and_rag 코드 04 (1/3)

이제 09_custom_workflow_and_rag 코드 04 (1/3) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 클래스 RouterState는 데이터 구조나 정책 단위를 정의합니다. 함수 classify, handle_math, handle_language, handle_general는 실제 작업 단위입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

255. 09_custom_workflow_and_rag 코드 04 (2/3)

이제 09_custom_workflow_and_rag 코드 04 (2/3) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 함수 route는 실제 작업 단위입니다. 핵심 API는 StateGraph, compile, add_node, add_edge, add_conditional_edges입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

256. 09_custom_workflow_and_rag 코드 04 (3/3)

이제 09_custom_workflow_and_rag 코드 04 (3/3) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

257. 09_custom_workflow_and_rag 코드 05 (1/2)

이제 09_custom_workflow_and_rag 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 graph에서 MessagesState를 준비합니다. 데코레이터 tool, tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 web_search, format_report는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

258. 09_custom_workflow_and_rag 코드 05 (2/2)

이제 09_custom_workflow_and_rag 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 그래프 실행의 기본 골격을 다룹니다. 상태 스키마, 노드 함수, 엣지 연결, compile 순서가 실행 모델의 중심입니다. 읽는 순서는 이렇습니다. 핵심 API는 StateGraph, compile, invoke, add_node, add_edge입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

259. 09_custom_workflow_and_rag 코드 06 (1/2)

이제 09_custom_workflow_and_rag 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_text_splitters에서 RecursiveCharacterTextSplitter를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

260. 09_custom_workflow_and_rag 코드 06 (2/2)

이제 09_custom_workflow_and_rag 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 15이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

261. 09_custom_workflow_and_rag 코드 07 (1/2)

이제 09_custom_workflow_and_rag 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 simple_search, retrieve_info는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

262. 09_custom_workflow_and_rag 코드 07 (2/2)

이제 09_custom_workflow_and_rag 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 9줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

263. 09_custom_workflow_and_rag 코드 08

이제 09_custom_workflow_and_rag 코드 08 슬라이드입니다. 노트북 위치는 09_custom_workflow_and_rag.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 검색 결과를 모델 답변의 근거로 연결하는 흐름을 다룹니다. 검색, 문서 구성, 출처 보존을 함께 봅니다. 읽는 순서는 이렇습니다. 먼저 import는 langchain_openai에서 OpenAIEmbeddings; vectorstores에서 FAISS를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

264. 10_production.ipynb

이제 CODE 10_production.ipynb로 넘어갑니다. 10_production.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/10_production.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

265. 10_production 코드 01

이제 10_production 코드 01 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 create_agent, ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

266. 10_production 코드 02 (1/2)

이제 10_production 코드 02 (1/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

267. 10_production 코드 02 (2/2)

이제 10_production 코드 02 (2/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

268. 10_production 코드 03

이제 10_production 코드 03 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 14줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 json를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

269. 10_production 코드 04 (1/2)

이제 10_production 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 language_models에서 GenericFakeChatModel; messages에서 AIMessage를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 get_capital는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

270. 10_production 코드 04 (2/2)

이제 10_production 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 5줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

271. 10_production 코드 05 (1/2)

이제 10_production 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 test_agent_trajectory는 실제 작업 단위입니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

272. 10_production 코드 05 (2/2)

이제 10_production 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 예외 처리 블록은 실패를 숨기기 위한 장치가 아니라, 실패했을 때 어떤 경로로 복구할지 정하는 장치입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

273. 10_production 코드 06

이제 10_production 코드 06 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 match에서 create_trajectory_match_evaluator; messages에서 HumanMessage, AIMessage, ToolMessage를 준비합니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

274. 10_production 코드 07 (1/2)

이제 10_production 코드 07 (1/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

275. 10_production 코드 07 (2/2)

이제 10_production 코드 07 (2/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

276. 10_production 코드 08

이제 10_production 코드 08 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 1줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

277. 10_production 코드 09 (1/2)

이제 10_production 코드 09 (1/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 fastapi에서 FastAPI를 준비합니다. 데코레이터 app.post가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

278. 10_production 코드 09 (2/2)

이제 10_production 코드 09 (2/2) 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 4줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 함수 chat는 실제 작업 단위입니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

279. 10_production 코드 10

이제 10_production 코드 10 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 먼저 import는 langgraph_sdk에서 get_sync_client를 준비합니다. 핵심 API는 stream입니다. 설정값 thread_id는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점

입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

280. 10_production 코드 11

이제 10_production 코드 11 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 19이고, 화면에는 실제 실행 코드 중 13줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

281. 10_production 코드 12

이제 10_production 코드 12 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 21이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

282. 10_production 코드 13

이제 10_production 코드 13 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 22이고, 화면에는 실제 실행 코드 중 12줄이 보입니다. 이 코드는 안전 정책을 에이전트 실행 경계에 넣는 방법을 다룹니다. 개인정보와 위험 응답은 모델 이후가 아니라 실행 중간에서 다룹니다. 읽는 순서는 이렇습니다. 먼저 import는 langsmith; langsmith를 준비합니다. 핵심 API는 invoke입니다. 설정값 context는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

283. 10_production 코드 14

이제 10_production 코드 14 슬라이드입니다. 노트북 위치는 10_production.ipynb의 cell 23이고, 화면에는 실제 실행 코드 중 11줄이 보입니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

284. 11_mcp.ipynb

이제 CODE 11_mcp.ipynb로 넘어갑니다. 11_mcp.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/11_mcp.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

285. 11_mcp 코드 01

이제 11_mcp 코드 01 슬라이드입니다. 노트북 위치는 11_mcp.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

286. 11_mcp 코드 02 (1/2)

이제 11_mcp 코드 02 (1/2) 슬라이드입니다. 노트북 위치는 11_mcp.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

287. 11_mcp 코드 02 (2/2)

이제 11_mcp 코드 02 (2/2) 슬라이드입니다. 노트북 위치는 11_mcp.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

288. 11_mcp 코드 03

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

289. 11_mcp 코드 04

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

290. 11_mcp 코드 05

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

291. 11_mcp 코드 06 (1/3)

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

292. 11_mcp 코드 06 (2/3)

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

293. 11_mcp 코드 06 (3/3)

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

294. 11_mcp 코드 07

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

295. 11_mcp 코드 08

이 Elicitation 부록 코드는 현재 노트북 cell 21과 같은 answer object 계약입니다. first.interrupts의 payload에서 requests[0].key를 꺼내고, Command resume에는 responses 딕셔너리를 넣습니다. 서버는 InputRequiredResult 호환 흐름을 전제로 둡니다.

296. 11_mcp 코드 09

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

297. 11_mcp 코드 10

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

298. 11_mcp 코드 11

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

299. 11_mcp 코드 12

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

300. 11_mcp 코드 13

이 11_mcp 부록 코드는 현재 노트북 셀과 같은 MCPAdapter 수명 주기를 기준으로 읽습니다. FastMCP Client와 ClientGroup은 인증을 소유하고, MCPConfig의 mcpServers는 다중 서버 target을 구성합니다. Elicitation은 interrupt payload의 request key에 대한 responses 객체로 재개합니다.

301. 12_frontend_streaming.ipynb

이제 CODE 12_frontend_streaming.ipynb로 넘어갑니다. 12_frontend_streaming.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/12_frontend_streaming.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

302. 12_frontend_streaming 코드 01

이제 12_frontend_streaming 코드 01 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실습 환경을 고정하는 준비 단계입니다. 단순 import가 아니라 이후 슬라이드에서 반복해서 쓸 실행 단위를 잡는 과정입니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

303. 12_frontend_streaming 코드 02 (1/2)

이제 12_frontend_streaming 코드 02 (1/2) 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

304. 12_frontend_streaming 코드 02 (2/2)

이제 12_frontend_streaming 코드 02 (2/2) 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 2줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

305. 12_frontend_streaming 코드 03

이제 12_frontend_streaming 코드 03 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 7이고, 화면에는 실제 실행 코드 중 8줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

306. 12_frontend_streaming 코드 04

이제 12_frontend_streaming 코드 04 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 9이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 asyncio를 준비합니다. 함수 stream_events_demo는 실제 작업 단위입니다. 핵심 API는 astream_events입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

307. 12_frontend_streaming 코드 05 (1/2)

이제 12_frontend_streaming 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 metadata에서 version; agents에서 create_agent를 준비합니다. 함수 get_weather는 실제 작업 단위입니다. 핵심 API는 create_agent, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

308. 12_frontend_streaming 코드 05 (2/2)

이제 12_frontend_streaming 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 11이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

309. 12_frontend_streaming 코드 06 (1/2)

이제 12_frontend_streaming 코드 06 (1/2) 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 먼저 import는 tools에서 tool; types에서 ToolRuntime를 준비합니다. 데코레이터 tool가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 analyze_data는 실제 작업 단위입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

310. 12_frontend_streaming 코드 06 (2/2)

이제 12_frontend_streaming 코드 06 (2/2) 슬라이드입니다. 노트북 위치는 12_frontend_streaming.ipynb의 cell 17이고, 화면에는 실제 실행 코드 중 7줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

311. 13_guardrails.ipynb

이제 CODE 13_guardrails.ipynb로 넘어갑니다. 13_guardrails.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습하도록 구성했습니다. 이 파트에서는 화면의 개념을 먼저 잡고, 이어지는 코드에서 같은 구조가 어떤 객체와 함수 호출로 표현되는지 확인합니다. 실습 파일은 02_langchain/13_guardrails.ipynb입니다. 슬라이드만 넘기지 말고 같은 이름의 노트북 셀과 함께 보면 코드의 위치가 분명해집니다. 여기부터는 코드 워크스루입니다. 각 슬라이드는 노트북의 실제 실행 셀을 기준으로 만들었고, 긴 셀은 읽기 쉽도록 여러 장으로 나눴습니다.

---

312. 13_guardrails 코드 01

이제 13_guardrails 코드 01 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 4이고, 화면에는 실제 실행 코드 중 10줄이 보입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 dotenv에서 load_dotenv; langchain_openai에서 ChatOpenAI를 준비합니다. 핵심 API는 create_agent, ChatOpenAI입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

---

313. 13_guardrails 코드 02

이제 13_guardrails 코드 02 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 5이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 이 코드는 실행 과정을 한 번에 기다리지 않고 이벤트 단위로 흘려보내는 방법을 다룹니다. 사용자 경험과 디버깅 모두에서 필요한 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 os를 준비합니다. 핵심 API는 invoke, stream입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 환경 변수와 토큰이 보이면 권한 경계를 함께 설명해야 합니다. 예제에서는 편의를 위해 노출되지만 실제 서비스에서는 비밀값 관리와 접근 범위가 따로 필요합니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

314. 13_guardrails 코드 03 (1/2)

이제 13_guardrails 코드 03 (1/2) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; middleware에서 PIIMiddleware를 준비합니다. 핵심 API는 create_agent, PIIMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

315. 13_guardrails 코드 03 (2/2)

이제 13_guardrails 코드 03 (2/2) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 8이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 PIIMiddleware입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

316. 13_guardrails 코드 04 (1/2)

이제 13_guardrails 코드 04 (1/2) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 21줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent; middleware에서 HumanInTheLoopMiddleware를 준비합니다. 핵심 API는 create_agent, InMemorySaver, HumanInTheLoopMiddleware, Command입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

317. 13_guardrails 코드 04 (2/2)

이제 13_guardrails 코드 04 (2/2) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 10이고, 화면에는 실제 실행 코드 중 16줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 대화 상태를 저장하고 다시 불러오는 구조를 다룹니다. thread_id와 checkpointer가 함께 있어야 같은 사용자의 흐름이 이어집니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke, Command입니다. 설정값 thread_id, configurable는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

318. 13_guardrails 코드 05 (1/2)

이제 13_guardrails 코드 05 (1/2) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 runtime에서 Runtime; typing에서 Any를 준비합니다. 클래스 ContentFilterMiddleware는 데이터 구조나 정책 단위를 정의합니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

319. 13_guardrails 코드 05 (2/2)

이제 13_guardrails 코드 05 (2/2) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 12이고, 화면에는 실제 실행 코드 중 18줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

320. 13_guardrails 코드 06 (1/3)

이제 13_guardrails 코드 06 (1/3) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 19줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 runtime에서 Runtime; messages에서 AIMessage를 준비합니다. 클래스 SafetyGuardrailMiddleware는 데이터 구조나 정책 단위를 정의합니다. 핵심 API는 init_chat_model입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

321. 13_guardrails 코드 06 (2/3)

이제 13_guardrails 코드 06 (2/3) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 17줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. 읽는 순서는 이렇습니다. 핵심 API는 invoke입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

322. 13_guardrails 코드 06 (3/3)

이제 13_guardrails 코드 06 (3/3) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 14이고, 화면에는 실제 실행 코드 중 3줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 노트북의 실행 흐름을 그대로 다룹니다. 결과값만 보지 말고 어떤 객체가 만들어지고 다음 단계로 무엇을 넘기는지 봐야 합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

323. 13_guardrails 코드 07 (1/3)

이제 13_guardrails 코드 07 (1/3) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 3장으로 나눠 본 첫 부분입니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 먼저 import는 runtime에서 Runtime; typing에서 Any를 준비합니다. 데코레이터 before_agent가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 content_filter는 실제 작업 단위입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

324. 13_guardrails 코드 07 (2/3)

이제 13_guardrails 코드 07 (2/3) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 데코레이터 after_agent가 일반 함수를 프레임워크가 인식하는 실행 단위로 바꿉니다 함수 safety_check는 실제 작업 단위입니다. 핵심 API는 create_agent입니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

325. 13_guardrails 코드 07 (3/3)

이제 13_guardrails 코드 07 (3/3) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 16이고, 화면에는 실제 실행 코드 중 6줄이 보입니다. 앞 슬라이드에서 이어지는 3번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 일반 함수를 모델 호출 도구로 바꾸는 부분입니다. 함수 이름, 인자 타입, docstring이 모델의 선택 근거가 됩니다. 읽는 순서는 이렇습니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

326. 13_guardrails 코드 08 (1/2)

이제 13_guardrails 코드 08 (1/2) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 20줄이 보입니다. 긴 코드 셀을 2장으로 나눠 본 첫 부분입니다. 이 코드는 에이전트 조립 표면을 다룹니다. 모델, 도구, 프롬프트, 미들웨어가 만나는 지점을 중심으로 읽습니다. 읽는 순서는 이렇습니다. 먼저 import는 agents에서 create_agent를 준비합니다. 핵심 API는 create_agent, HumanInTheLoopMiddleware, PIIMiddleware입니다. 설정값 middleware는 실행 결과보다 운영 경계를 설명합니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

327. 13_guardrails 코드 08 (2/2)

이제 13_guardrails 코드 08 (2/2) 슬라이드입니다. 노트북 위치는 13_guardrails.ipynb의 cell 18이고, 화면에는 실제 실행 코드 중 15줄이 보입니다. 앞 슬라이드에서 이어지는 2번째 부분입니다. 이전에 만든 객체가 여기서 어떻게 사용되는지 확인합니다. 이 코드는 실행 중간에 정책을 삽입하는 방법을 다룹니다. 미들웨어는 로깅, 동적 프롬프트, 재시도, 승인, 가드레일을 코드 흐름 안으로 넣는 장치입니다. 읽는 순서는 이렇습니다. 핵심 API는 HumanInTheLoopMiddleware입니다. print 출력은 학습용 확인 지점입니다. 운영 코드에서는 같은 위치가 로그, trace, 평가 지표로 바뀐다고 생각하면 됩니다. 이 슬라이드에서는 결과값보다 입력, 중간 객체, 다음 단계로 넘어가는 값을 먼저 봅니다.

328. LangChain은 빠른 조립 도구가 아니라 운영 가능한 에이전트 인터페이스입니다

LangChain은 빠른 조립 도구가 아니라 운영 가능한 에이전트 인터페이스입니다. create_agent에서 시작해 메시지·도구·메모리·미들웨어·HITL·MCP·UI·가드레일까지 연결하면, 다음 LangGraph 트랙에서 직접 그래프를 설계할 준비가 끝납니다. 이 트랙에서 남겨야 할 핵심은 create_agent를 중심으로 모델, 도구, 메시지, 메모리, 미들웨어, MCP, 가드레일을 하나의 운영 가능한 에이전트 인터페이스로 연결하는 일입니다. 실습 코드를 모두 외울 필요는 없습니다. 대신 어떤 상황에서 어떤 레이어를 선택하고, 상태와 도구와 권한을 어디에서 관리할지 말로 설명하는 수준까지 가야 합니다. 이 기준으로 다시 노트북을 열어 보면, 각 코드 셀이 단순 예제가 아니라 운영 가능한 에이전트를 구성하는 하나의 결정 지점으로 보입니다.